Evidence Before Impact: Upstream Change Intelligence for Dependency Decisions

Mehmet KIŞ
Ikarus AI Labs, Cambridge, United Kingdom*
mehmet@ikaruslabs.net https://ikaruslabs.net/

Technical Report TR-2026-01 31 October 2026 OpenPulse v0.8.0

Abstract

Dependency decisions rely on vulnerability advisories and lifecycle catalogues, yet several upstream changes that alter a dependency's trust, maintenance or availability appear in neither: a repository relocated or archived, or versioned images withdrawn from a namespace still in use. AI coding agents now make such decisions too. The central risk is a *false clear*: reporting a dependency as not affected when it is. We present OpenPulse, an open-source instrument that collects, qualifies and applies upstream change evidence to declared inventories. Completeness-aware negatives, conservative abstention and a third truth value are established ideas. Our contribution is a specific, testable model that combines them for dependency applicability: typed scope elements over heterogeneous identities, completeness with a recorded basis, and a negative only when a complete scope has every element decisively excluded, with a short conditional soundness lemma. An audit against this model found sixteen defects, thirteen of which could produce or display false clears, some encoded as correct by the regression suite; fifteen are fixed and one is open. An oracle written from the specification agrees with the implementation on 100,000 random cases and a 6,195-case matrix, apart from one documented, non-blocking label on 72 cases, and catches all 22 historical-defect and rule-disabling mutants. An exploratory retrospective study used 27 historical changes, selected under quotas, and 167 dependency references labelled by the developer; it describes those events, not a deployed rate. Of 103 affected references, a naive reading falsely cleared 35 in 10 events and the event-scope engine none. The engine warned on 64, left 27 unresolved and missed 12 silently. The unresolved include all 16 compromised-package references, which a minimal conservative baseline flagged; the engine had no curated identity for them. Typed membership and identity rules, not the completeness condition, account for the false-clear difference; the condition rests on constructed cases and conformance. Only if the events were a random sample would zero in 27 bound the per-event rate below 0.105. The study's follow-up found a false clear in the advisory path, which the study did not run; figures re-measured after the fixes are diagnostic. The security claim is conditional and covers one decision path, event-scope applicability: given faithfully constructed, current events, correct completeness declarations, correct identity relations and, for a summary, every relevant event loaded, the engine issues no negative inconsistent with those inputs. Whether they deserve that trust is unmeasured, and the claim does not extend to the advisory path, to verdict consumption or to agent behaviour. A prospective, independently labelled evaluation is specified, with its analysis implemented, as an open protocol (Appendix F).

*Ikarus AI Labs is a non-profit research initiative based in Cambridge, UK, working on evidence-based assurance for AI-mediated software change. We are open to collaboration on the evaluation and integrations described in this report: https://ikaruslabs.net/.

1 Introduction

Modern applications consist largely of third-party code, and each dependency carries assumptions about its upstream: who controls it, where it is published, whether it is maintained, and how long its release line is supported. Dependency decisions check those assumptions mostly against curated records. Software composition analysis (SCA) matches inventories against vulnerability databases such as NVD and OSV, and lifecycle tools consult catalogues such as endoflife.date. Several consequential changes appear in neither. Three examples from our own observations:

- **Namespace relocation.** The repository at `github.com/coreos/etcd` now resolves to `github.com/etcd-io/etcd`. References that record the old path keep working through a platform redirect, but they no longer name the canonical location, and their trust assumptions now depend on the redirect persisting.
- **Archival.** A repository marked archived receives no fixes *in that repository* while it stays archived.
- **Distribution withdrawal.** In 2025 Bitnami announced that versioned images would move from `docker.io/bitnami` to an unsupported `docker.io/bitnamilegacy` namespace [3]. In October 2026 the former mainline repository `docker.io/bitnami/spark` listed no tags.

Detecting such a change is not enough. A dependency decision needs to know whether *this* dependency is within the change's scope, how well that is established, and what remains unknown. That step, *applicability*, has a security-relevant failure mode: the *false clear*, in which a dependency is reported as not affected when it is in scope. A false clear looks exactly like a correct negative. Name-based matching conflates `docker.io/redis` with `docker.io/bitnami/redis`, and an absent match is often read as an absent problem.

The question matters more as AI coding agents propose dependencies. Language models hallucinate package names that attackers can register [53]. Merely checking that a package exists does not defend against that, because an attacker-registered name passes an existence check. More generally, delegating dependency choices to agents creates an additional automated decision surface. How fast that surface grows, and what incident burden it carries, are empirical questions we do not answer. Our position is narrower: dependency existence and model confidence are insufficient, and current, scoped ecosystem evidence is an additional input to the decision.

Thesis and questions. OpenPulse investigates how current upstream change evidence can inform dependency decisions made by developers and AI coding agents, while preventing incomplete evidence from becoming a false clearance. Its claim is narrower than that motivation. OpenPulse is an evidence and applicability layer that could support agent-mediated dependency decisions; it is not a validated defence against malicious packages, prompt injection or unsafe agent actions, and no agent experiment has been run. Its assurance is conditional: given faithfully constructed, current events, correct completeness declarations, correct identity relations and, for a summary, every relevant event loaded, the engine issues no negative inconsistent with those inputs, and none of these inputs is measured yet (Table 19). The report is organised around three questions:

Q1. What upstream change evidence is missing from a dependency decision, and at which decision points? (Section 3)

Q2. How can that evidence be collected, qualified, and applied to a dependency without unsound negatives? (Sections 4–7)

Q3. What measurable difference does it make? (Section 10; protocol in Section 9)

Contributions.

1. A **typed applicability model**: membership per scope-element kind, scope completeness with a recorded basis, exact elements kept apart from envelopes, and identity, freshness and consistency in one admission rule for negatives, with two short conditional lemmas and a decomposition of their premises (Section 5). Section 2.1 separates prior work from this addition. Its most distinctive part, the completeness condition, prevented no false clear in the retrospective study.

2. An **implementation audit** of OpenPulse against the model. It found sixteen defects; thirteen could produce or display false clears, including expectations the regression suite had encoded as correct. Fifteen are fixed and one (D16) is open, and the admission conditions are enforced with the gaps listed in Section 11 (Section 6).

3. **Conformance and differential evidence**: an oracle written from the specification, a generated matrix, 100,000 randomised cases, 22 re-injected mutants and differential version testing against four ecosystems' reference implementations, run as a build gate (Section 6).

4. An **exploratory retrospective study** on 27 historical changes, selected under quotas, and 167 real dependency references, frozen before any policy ran, with the developer's labels. It reports false clears, warnings, unresolved references and silent misses side by side (Section 10).

A prospective, independently labelled evaluation is specified, and its analysis code implemented, as an open protocol (Section 9 and Appendix F).

Status. Table 1 separates what is specified, what is implemented and what is open. Every statement about behaviour, and every measurement, refers to one release, OpenPulse v0.8.0 (git tag `v0.8.0` of `https://github.com/IkarusAILabs/OpenPulse`), under the Apache License 2.0. Earlier states appear only where the method needs them: the audited commit and the audit snapshot in Section 6, and the retrospective study's freeze tag in Section 10. The artifact manifest (Appendix C) gives the commands and expected outputs, and lists the changes made after the audit. Limitations are collected in Section 11 and are not repeated.

Terminology. A *false clear* is a NotAffected verdict for a dependency in the event's scope; a *missed detection* is an upstream change that discovery never reports. Event confidence grades the evidence for an event, and verdict confidence grades one dependency's verdict. *Complete* has three senses that do not imply one another: acquisition coverage, scope completeness and inspection coverage. An event's *asserted* scope is what its sources say; its *established* scope is the part whose completeness has a recorded basis; and its *evaluable* scope is the part OpenPulse can test with the semantics it implements. A negative needs the whole asserted scope to be established and evaluable. Unsupported or unevaluable elements are kept and give `?`, which blocks a clearance. The envelope is never narrowed to the subset the implementation can evaluate. *Artefact* means a distributable object, and `artifact` is its spelling in code identifiers.

Component	Specified	Implemented at v0.8.0	Open
Scope typing	Repository, namespace, tag, release-line, object, package, registry, project elements, each *exact* or *envelope*	Artefact elements with roles by form; tag predicates `<version>` and `*`	Roles not labelled in the contract
Canonical form	Per element kind, shared with the oracle	Lowercase; scheme, tag, digest stripped; `docker.io` default; Docker Hub host aliases folded; package names lowercased, PEP 503 for PyPI, Maven coordinates; purl values by ecosystem and name	Other package-name forms (npm scopes, Go module paths): unequal names undetermined
Completeness	Verified / declared / partial	Opt-in declaration; unspecified is partial	Declarations are not verified
Aggregation	Every element evaluated; negative needs complete scope	Yes; regression- and conformance-tested	–
Cross-event summary	No negative masks an unresolved association	Summary step, checked exhaustively; unmapped identity is RELATED	–
Versions, OSV ranges	Union of intervals; unsupported forms undetermined	Yes; each range compared under its own scheme (SemVer, PEP 440, plain numeric)	Differential against `packaging` (PEP 440), and in CI against the pinned npm, Go and Maven tools; others untested
OCI digests	Directional, platform-aware containment	Interim: unequal digests undetermined	Level resolution; no digest-based negatives meanwhile
Freshness	Negatives only from current, non-superseded evidence, per dependency	Withdrawn and superseded events dropped; one 90-day bound on event-scope negatives over the newest evidence, default in the CLI and agent tools, opt-in in the library call	Per-class bounds; failed acquisition does not withdraw; advisory negatives unbounded; a past `-as-of` admits later evidence
Contract	Evidence, unknowns, negative semantics	1.2.0, unsigned; verdict meaning and completeness	Supersession, revocation, expiry
First detection	Externally witnessed	Unkeyed local ledger	Backdating not detectable

Table 1: Specification versus implementation at release v0.8.0. *Implemented*: present at the release and covered by regression tests. *Interim*: a fail-closed substitute that withholds negatives where the specified semantics are missing (Table 15). *Open*: specified but not implemented, or not established.

2 Background and Related Work

Abandonment and deprecation. Empirical work studies how projects go dormant [57], how they survive the loss of core developers [2], how practitioners perceive the bus factor [25], and how npm packages are deprecated [7]. Developers lack systematic ways to learn that a dependency was abandoned [35]. These studies characterise the process. We detect observable end states and relate them to inventories.

Dependency management. Dependency networks differ in size, growth and update practice across packaging ecosystems [13], and developers often keep outdated dependencies, frequently

because they are unaware that an update or a migration is needed [28]. That unawareness is the gap this report addresses for changes that are not vulnerabilities: the question is not whether to update, but whether a given upstream change applies to a given declared dependency at all.

Supply-chain attacks and namespaces. Attack taxonomies [40, 29] and ecosystem studies [62, 60, 22] identify account takeover, abandoned projects and namespace reuse as vectors. GitHub documents that transfers leave redirects, that creating a repository at the old location removes them, and that some names are retired against reuse [18]. Industry research describes the resulting "repojacking" risk [19].

Applicability and vulnerability data. Vulnerability applicability has been studied at the level of code: impact assessment and reachability analysis decide whether vulnerable code is present and used [45, 46, 47]. That is a finer question than ours, and a complementary one. It assumes that the vulnerable component has already been associated with the dependency, which is the step at which our false clears arise. Naive counting inflates vulnerability figures [43], and SCA tools disagree [24]. OSV [41], GHSA [17], NVD [37], KEV [10] and endoflife.date [15] are the dominant feeds. NVD data has known inconsistencies [1], and prior work measures lags between events and records [33, 4].

Provenance, posture and logs. in-toto [56], SLSA [52] and Sigstore [38] establish build provenance. CycloneDX [14] and SPDX [55] standardise inventories. Empirical studies of SBOM practice report that producers and consumers struggle with the completeness and accuracy of the inventories themselves [58, 54]. Our setting has the same problem on the other side of the match: an inventory may be incomplete, and so may an event's affected set. Only the second is modelled here; inventory coverage is reported as a separate qualifier (Section 7). Scorecard [59] and deps.dev [20] provide posture signals. Hash-chained logs [50], history trees [9] and Certificate Transparency [30, 31] are the standard tamper-evident designs.

AI agents and containers. AI-assisted code is often insecure [44]. Models hallucinate package names [53, 27], and agents that read external content are exposed to indirect prompt injection [21, 12]. This work studies whether a recommended package exists or is malicious. It does not ask whether a real, existing dependency is affected by an upstream change after the model's knowledge was fixed, which is the input our model supplies to such a decision. Container studies examine image content [51, 61, 5, 34] rather than changes to how pinned images are distributed.

Completeness reasoning. The underlying principle is old. Under the closed-world assumption, absence from a database is read as falsity [49]. Without it, a negative conclusion needs explicit knowledge of completeness. A third truth value for "unknown" goes back to Kleene's strong three-valued logic [26] and to null values in the relational model [6], and incomplete-information databases make the distinction formal [23]. Completeness assertions attached to parts of a database say where absence may be read as falsity [36]. Local completeness statements let a query answer be declared complete from incomplete sources [32], and later work decides when a query over partially complete data is itself complete [48]. Local closed-world reasoning lets a planner draw negative conclusions only where its information is known to be complete [16]. Our coverage premise is an over-approximation in the sense of abstract interpretation [8]: sound for exclusion, and silent about membership. We claim no new logical principle. Our contribution is to apply a local completeness assertion to the affected set of an upstream change: a typed representation over heterogeneous dependency identities, an implementation discipline and an audit against it. Whether it improves decisions is the empirical question of Section 9.

Representation and consumer policy	`bitnami/redis:7.2`	`bitnami/nginx`	`docker.io/redis`
(1) Affected list of five images; absence read as not affected	affected	not affected (false clear)	not affected
(2) Same list as VEX; unlisted treated as under investigation	affected	under investigation	under investigation
(3) Namespace elements, χ declared (announced basis)	affected	affected	not affected

Table 2: One event, three representations. Ground truth: both Bitnami images are in scope, and `docker.io/redis` is not.

Closest semantics. VEX [11, 42] is the closest representation of applicability. It has an explicit unresolved status (*under investigation*). The CISA minimum requirements attach a justification or impact statement to *not affected*, which preserves a rationale for a negative. VEX documents are versioned, and we have not verified how consistently consumers invalidate earlier statements. CSAF [39] carries the same statuses (known affected, known not affected, fixed, under investigation) over a product tree whose branches can name vendors, products and version ranges; status is still asserted per product in that tree. Two properties we did not find in the formats we reviewed (OSV, VEX and OpenVEX, CSAF, CycloneDX): a representation of whether an *affected-object set is complete*, so that a negative can be conditioned on it, and one applicability rule relating an upstream change to *heterogeneous* dependency identities. This is a statement about those formats, not about every tool. Table 3 compares *stated design scope*. It is not a benchmark.

The contribution is a *property* and an *encoding* of it, not a new data format. The property is that a negative verdict is conditional on a stated completeness of the affected set. The encoding is the pair (E, χ) with typed elements. Existing formats can approximate it. A VEX producer can list every product with an explicit status. A consumer policy can treat every unlisted product as *under investigation*. A justification field can carry a free-text claim of completeness. What none of these provides is a machine-readable statement, attached to the affected set, that tells the consumer whether absence from the list may be read as exclusion.

Table 2 works one case through three representations: Bitnami's catalogue-wide change, with three dependencies. (1) A conventional affected-product list of five images, read with absence as not affected, clears `docker.io/bitnami/nginx`. That is a false clear, and it is defect D4 in our own implementation (Section 6). (2) The same list, as VEX statements consumed under a conservative policy that treats unlisted products as under investigation, is safe. But it cannot clear `docker.io/redis`, which is plainly outside the change, unless the producer enumerates every unaffected image in existence. (3) With typed elements and χ, the namespace element declared complete makes `bitnami/nginx` affected and clears `docker.io/redis`. The same five-element list with χ partial behaves like (2). The difference between (2) and (3) is decisiveness at equal safety, not safety. The existing-representation baseline of Section 9, given the same identity knowledge (X_K in Table 21), measures that difference. The strong conservative baseline (v) does not, because it already has (3). What (3) makes machine-checkable is that the license to read absence as exclusion travels with the affected set, as data with a recorded basis, instead of living in each consumer's policy. A conservative consumer policy remains expressible in existing formats. We have not found a way to state "every image under this namespace" with the product identifiers of the formats we reviewed. Where that is possible, (2) can approach (3).

2.1 What is new, and what it enables

Completeness-aware reasoning, conservative abstention and a third truth value for "unknown" are established ideas (above), and we claim none of them. Our claim is narrower and testable: a

Approach family	Disc.	Ident.	Appl.	Unkn.	Temp.	Export
Vulnerability feeds (OSV, GHSA, NVD)	–	P	P	–	P	Y
Lifecycle catalogues (endoflife.date)	–	P	P	–	Y	P
SCA tools	–	P	Y	P	–	P
Reachability analysis [46]	–	P	Y	P	–	–
VEX [11, 42]	–	P	Y	Y	P	Y
Posture (Scorecard, deps.dev)	P	P	–	–	–	P
Provenance (in-toto, SLSA, Sigstore)	–	Y	–	–	P	Y
OpenPulse (this report)	P	Y	Y*	Y	P	P

Table 3: Stated design scope, taken from each family's documentation; not a measured comparison. Disc.: discovery of non-lifecycle upstream change. Ident.: identity across namespaces and pinning forms. Appl.: event-specific applicability. Unkn.: explicit unknown. Temp.: provenance of dates. Export: verifiable machine-readable output. Y: primary goal; P: partial; –: outside stated scope (absence not tested). *Implementation gaps in Table 1. The families are of different kinds and are not interchangeable: OSV, VEX, CycloneDX and SPDX are data formats; NVD, GHSA, KEV and endoflife.date are curated feeds; SCA tools and reachability analysis are consumers that apply a policy; Scorecard and deps.dev are posture signals; and in-toto, SLSA and Sigstore are provenance and build assurance. The text after this table gives a worked example of the behavioural difference.

specific *model for dependency applicability decisions*, made precise enough that an implementation can be checked against it and that it can be compared with simpler policies on real changes. Table 4 separates, for each part of that model, what is prior, what this report adds, how the addition is tested, and what the evidence shows so far. Each row is a candidate contribution, and several are not yet validated.

What the model lets one do. The question a reader should ask is what a researcher or practitioner can do with this model that existing approaches do not do as reliably. Our answer has five parts, each with its evidence and its limit.

1. *Issue a negative whose justification travels with the data.* The completeness state and its basis are part of the affected set, so any consumer can mechanically reject a NotAffected whose scope is not complete on a recorded basis. In existing formats that rule lives in each consumer's policy (Table 2). The retrospective study did not exercise this rule: typed membership and identity avoid the naive reading's 35 false clears, and with completeness assumed everywhere the model still issued none (Section 10). This part is about making negatives auditable, not measured safety.

2. *Clear what is plainly out of scope without enumerating it.* Namespace, registry and release-line elements with a declared complete scope let a consumer clear `docker.io/redis` under a Bitnami change at equal safety (Table 2). Against a policy that never clears, the retrospective gain in resolution was 0.091 at the freeze (below the pre-specified 0.10) and 0.170 after two defect fixes, a post-hoc figure. That comparator is a floor; whether the model resolves more than existing formats read with the same identity knowledge is untested (Table 21).

3. *Keep identity uncertainty visible instead of turning it into exclusion.* The identity rule is the part with the clearest evidence: ablating it, so that an unresolved package or image counts as unrelated, silently dropped 18 more in-scope pairs in the retrospective study.

4. *Test an implementation against the decision rule.* Because the model fixes what absence means for each element kind, an oracle can be written from the text and compared with any implementation. Doing so found defects that the regression suite had encoded as correct

(Section 6), and every re-injected mutant is caught. Formats that leave absence to the consumer give no such reference to test against.

5. *Say what cannot be stated.* A scope that the language cannot express exactly is stated as an over-approximation, which can cost resolution or cause a false impact but never a false clear, and recorded as a gap, never silently narrowed. The retrospective study found four such forms, which bounds the model's reach rather than hiding it.

What the model does not do is equally specific. It does not decide whether vulnerable code is reachable. It does not by itself make decisions better than a careful implementation of the same model: the strong baseline (v) uses the same scope language and differed from OpenPulse by 0.030 in resolution at the freeze. So the contribution is the model and its checkability, not a superior engine. Evidence for the completeness condition itself needs in-scope references outside a stated partial list; the prospective evaluation reports D1 and the completeness ablation by basis, but does not seek such events out.

Candidate contribution	Established idea	What this report adds	Evidence so far
Representation of scope and its completeness	Completeness assertions on parts of a database [36, 32]	Typed scope elements over heterogeneous dependency identities (repository, tag, release line, object, package, registry, project), with completeness as a state plus a recorded basis, exported in the evidence contract (Sections 5.2 and 7)	Conformance: 100,000 random cases agree with an oracle; of 6,195 matrix cases, 6,123 conform and 72 differ by one non-blocking label, none unsafely. Representability: 27 real events; four scope forms could not be stated exactly (Section 10)
Conditions for a justified negative	Closed-world and local closed-world reasoning [49, 16]	Admission conditions N1–N5 and a lemma separating the aggregation argument from the premises an implementation must establish (Section 5.4)	Proof is conditional. Constructed cases and conformance only: on 27 events, assuming completeness everywhere gave no false clear and changed 7 decisions, all correct negatives the condition withheld (Section 10)
Exact elements versus envelopes	Over-approximation in abstract interpretation [8]	Roles by element form: an envelope may exclude but never yields a positive; an exact element may do both (Section 5.2)	The over-match D7 was found and fixed; the D7 and role mutants are caught. Retrospective false impact 1.6% against 3.2% for the list-based policies
Identity, freshness and consistency in one decision	Conservative handling of unknowns; VEX *under investigation* [11]	Unresolved identity never excludes (D10, F4); stale, withdrawn, superseded and contested events never clear (N4, N5); per-ecosystem package identity (F1–F3) (Sections 5.5 and 4.4)	Identity: treating unresolved identity as unrelated raised silent misses from 11.7% to 29.1% of in-scope pairs. Replay at the date of each event's newest evidence kept every event fresh, and no evidence was contested, so those rules were not exercised
Implementation, audit and evaluation	Differential and mutation testing; prospective evaluation	An oracle written from the specification, a defect re-injection suite, an open registration protocol with its analysis code implemented, and an exploratory retrospective study (Sections 6, 9 and 10)	22 of 22 historical-defect and rule-disabling mutants caught. Retrospective, on 27 selected events: no false clear where a naive list reading had 35 (10 events), but 39 of 103 in-scope references without a warning; descriptive, not a rate. Pre-specified test not yet run; labels not independent

Table 4: What is prior, what this report adds, and what is validated. "Evidence so far" states results and their limits; none of the rows is established by the prospective, independently labelled evaluation that Section 9 specifies.

3 Q1: What Evidence Is Missing from a Dependency Decision

Change classes. An *upstream change* alters the state of a project or its distribution channels in a way that can change the risk, operability or legal standing of dependents. OpenPulse *discovers* three classes: namespace relocation and archival on GitHub, and distribution change on Docker Hub. It watches catalogued projects, and also derives what to watch from the inventory (Go module paths, `github.com` references and Docker Hub images). End-of-life, end-of-support and vulnerability events come from existing feeds, and every npm, PyPI, Maven, Go and crates.io package is also queried on OSV by its own name. Licence and support-model changes can be represented and matched only as curated events.

Decision points. The evidence is relevant at three stages, and OpenPulse contributes evidence to each one without owning enforcement or choosing replacements:

1. *Before adding* a dependency: is the proposed identity what it appears to be, and what current upstream evidence exists about it?
2. *Before installing or executing* it: is there enough evidence, and enough approval, to proceed?
3. *After adoption*: which upstream changes now apply to the actual inventory?

Risks. Table 5 separates the risks that motivate the work from those the implemented evidence addresses.

Design constraints. (C1) No claim without named evidence, and weak evidence never reaches action-level channels. (C2) An undetermined question is never reported as a negative. (C3) Verdicts are deterministic functions of recorded inputs, with no language model in the decision path. (C4) Output is evidence and a recommendation to investigate. Nothing is remediated automatically.

Risk	Relation to OpenPulse
Invented or attacker-registered package names	Motivation only. Not a demonstrated OpenPulse capability; existence checks alone are insufficient [53].
Existing dependencies with stale maintenance, namespace or distribution assumptions	Directly addressed by the implemented change classes (Appendix D).
Prompt injection through dependency intelligence	Created by agent-facing consumption; acknowledged, untested (Section 7).

Table 5: Risks in AI-mediated dependency selection.

4 Q2a: Collecting and Qualifying Evidence

4.1 Pipeline and trust boundaries

Collectors acquire data from GitHub, OSV, NVD, MITRE CVE, CISA KEV, endoflife.date and the Docker Hub/OCI registry. Pure-function analysts derive *findings*. A gate vetoes events whose claims exceed their support. The verdict engine relates gated events to inventories. Renderers produce reports, alerts, evidence contracts and agent responses (Figure 1). A failed acquisition becomes a recorded gap, never an empty result.

4.2 Observation, claim, event

An *observation* records what a source exposed at time t. A *claim* is an assertion that names its supporting evidence. An *event* is a gated, scoped and dated unit of intelligence. A diff between observations is only a candidate change, with an explicit evidence strength, until it is gated.

4.3 Authority, independence and confidence

Sources are tiered as official, primary, secondary or tertiary. Authority is claim-specific in principle: GitHub is authoritative for a repository's canonical path, not for why it moved. Corroboration counts independent source families, folding derivation chains, and NVD and OSV count as one family. Where no family is declared, the implementation falls back to the source name, so undeclared copies of one origin can count as independent. Confidence (Unverified < Emerging < Corroborated < Confirmed) and impact (Informational < Watch < Review < Action < Critical) are *policy categories*, not calibrated probabilities. Table 6 collects every rule. A verdict's confidence is

$$\mathrm{conf}(d, e) = \min\big(\mathrm{ceil}(\mathrm{level}(d, e)),\ \mathrm{conf}(e),\ \mathrm{cap}_{\mathrm{id}}(d, e)\big). \tag{1}$$

Here $L = (\text{Unverified} < \text{Emerging} < \text{Corroborated} < \text{Confirmed})$ is a total order, and min is its meet. $\mathrm{conf}(e) \in L$ is the gated event confidence. ceil maps the verdict's match level to L by the "match ceiling" row of Table 6. $\mathrm{cap}_{\mathrm{id}}(d, e)$ is Emerging when the verdict is affected through a Review-Required or Unverified identity mapping, other than by exact artefact equality, and Confirmed (no effect) otherwise. The equation is normative, and it is also what the implementation computes (`core/risk/check.py`). Impact is copied from the gated event and never raised by the verdict engine. Equation (1) involves no numerical thresholds. It takes the minimum over an ordinal scale, so each cap can only lower a verdict's confidence. Neither confidence nor impact has a numerical interpretation, in outputs or in evaluation tables. The ceilings and caps are design choices, not calibrated values. Whether they separate reliable verdicts from uncertain ones is measured by the stratified error rates in Appendix F.

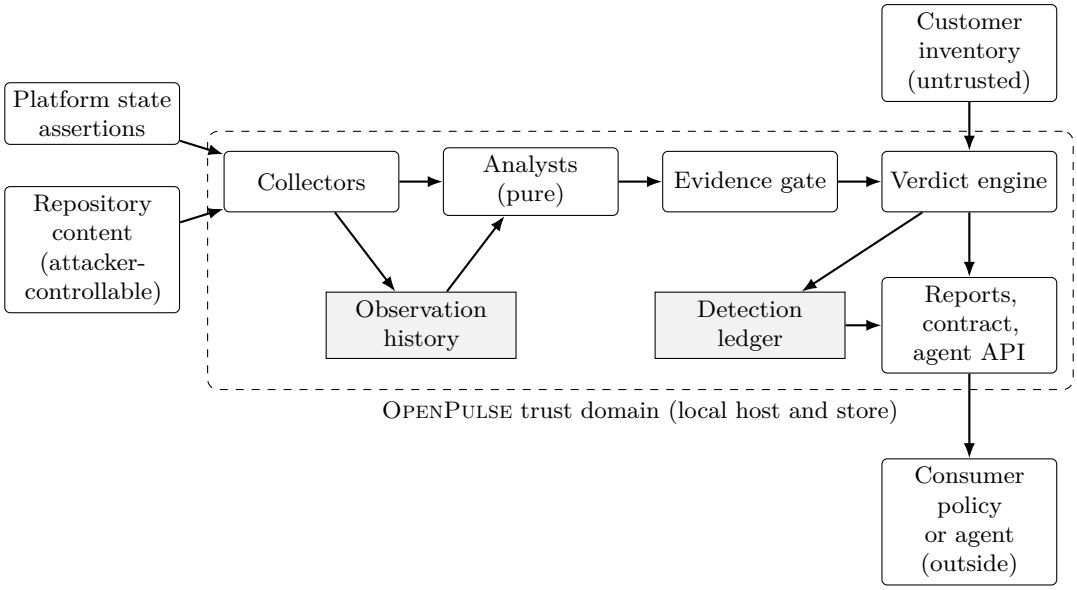


Figure 1: Pipeline and trust boundaries. Platforms are trusted to report their own *state* truthfully at query time: canonical path, archived flag, tag list. They are not trusted to be complete or current. Repository and advisory *content*, such as descriptions, release notes and names, is attacker-controllable and is treated as data, never as instructions. The stores protect only against accidental corruption. Policy is outside OpenPulse.

Stage	Rule
Event confidence	Confirmed: official evidence (official or primary for lifecycle classes). Corroborated: $\geq$2 independent families. Emerging: one credible source. Unverified: otherwise.
Gate	Unverified $\Rightarrow$ impact $\leq$ Watch; Emerging $\Rightarrow \leq$ Review; Action/Critical need $\geq$ Corroborated; artefact classes must name artefacts; dated classes need dates; unresolved contradiction blocks Action/Critical.
Vulnerabilities	Keyword-only match $\leq$ Review; KEV with undetermined applicability $\Rightarrow$ Review.
Match ceiling	object, repository, exact namespace or tag $\rightarrow$ Confirmed; version, not-affected $\rightarrow$ Corroborated; package, project $\rightarrow$ Emerging; related, unknown, and any envelope membership $\rightarrow$ Unverified.
Identity cap	Affected through a Review-Required or Unverified mapping $\Rightarrow \leq$ Emerging.

Table 6: All confidence and impact rules.

4.4 Temporal roles and lead time

Five roles are kept apart: *announced* (from sources with a trustworthy publication date, otherwise unknown), *effective*, *first detected*, *last verified* and *reported*. Conflicting effective dates are preserved as separate evidence. Detection lead is $\text{lead}(c) = \text{effective}(c) - \text{firstDetected}(c)$ in days. Positive means advance warning, negative means a measured late detection, and the lead is undefined only when a term is unknown. A lead depends on when monitoring started and how often it polls, so it is an *instrument capability and evaluation metric*, not an established benefit.

Freshness and supersession. A historical observation presented as current can mislead, for instance about where a repository now lives or whether a tag still resolves. We specify freshness per event class c and dependency role r with a policy bound $\tau_{c,r}$ (below). An observation is *fresh* while its last successful acquisition is within that bound of the evaluation time, and not later than it. A failed acquisition records a gap and never refreshes it. An event is *current* if

its *last verified* date is fresh and no later event from the same source corrects, withdraws or supersedes it. A negative verdict is admissible only from a current event. From a stale event the verdict is RELATED, with the reason *stale* kept distinct from *undetermined*, so that a consumer can tell old evidence from unresolved evidence. A withdrawn advisory asserts nothing.

In the implementation, verdicts are recomputed from the loaded events on every run, and nothing stores a negative for reuse, so a corrected event changes the next run's verdict. The contract exports `last_verified`. Withdrawn OSV entries are skipped, and withdrawn or superseded events are not evaluated. The bound is one maximum evidence age, not one per class. Given an evaluation time, a negative from an event whose *newest* evidence is older becomes RELATED, with the reason *stale*. This is weaker than the specification. One recently acquired item makes the whole event fresh, however old the others are, and the completeness declaration and the identity mapping carry no date that the check reads; two of the retrospective study's events (DIS-1, DIS-4) were fresh only through their newest item. The check applies to event-scope negatives only. Advisory-path negatives bypass it; this gap is open.

The command line's verdict commands (`check`, `attest`, `digest`, `warnings`) and the agent tools apply a 90-day bound by default, as of the start of the current day. `-replay`, or replay mode in the agent's Python API, drops the bound and says so, because replaying recorded evidence offline would otherwise make every negative stale. Contracts record the frame in `evaluation_context`. The library call (`check_dependency`) applies a bound only when the caller passes one. The 90-day value is a placeholder: in the retrospective study it changed no decision, and it is not calibrated to any event class. An evaluation time in the past (`-as-of`) is a reference point for the bound, not a knowledge cut. Evidence acquired after it is still used, so a past date can give a negative that nothing known at that date supported. This gap is open; the evaluation harness avoids it by cutting knowledge first. The report-placement freshness classes in `core/freshness.py` decide whether a change is *news*. They are not evidence currency and do not gate verdicts.

Three levels of currency are kept apart: (i) *evidence recently fetched*; (ii) *claims recently revalidated*, that is, re-derived from a re-acquired response whose upstream state is itself current; and (iii) *a verdict still valid when consumed*. By default the implementation establishes part of (i): the newest item of an event is no more than 90 days old. It establishes (ii) only for lifecycle events the agent derives live from a freshly fetched response, and even then not the upstream's own currency. Curated events, completeness declarations and identity mappings are never revalidated. It does not establish (iii): an exported contract has no expiry or supersession. Recomputation is not freshness either. A verdict recomputed today from an observation acquired weeks ago is as old as that observation, and the agent interface reuses each fetched response, successful or failed, for up to an hour (Section 7). If a source is unavailable, its acquisitions are recorded as gaps and the events already loaded stay in force; nothing withdraws their negatives (Section 11).

Event classes need different currency rules. A relocation is a historical fact. Its observation stays true of the time it was made, and what can go stale is whether the redirect still holds. Availability of a tag or image is a current state, and needs a short bound. A scheduled lifecycle statement is valid until its source revises it, and needs re-verification of the schedule, not of the date. Currency is also a property of what a negative *depends on*, not of the event as a whole: the evidence that defines the event's scope, the completeness declaration and the item supporting its basis, and, for an exclusion on identity grounds, the dependency's curated mapping (Section 5.5). Each must be fresh under $\tau_{c,r}$; a role may need only non-supersession, as for a historical relocation's announcement. The implementation does not record which item plays which role. Taking the oldest item instead is no substitute: in the retrospective study it would have withheld six correct negatives of the relocation events DIS-1 and DIS-4. There is also a time-of-check to time-of-use gap across the decision points of Section 3. A fresh answer before adoption says nothing about the state at installation. A bounded implementation therefore

binds each assessment to the exact reference, ideally a digest, and to its evaluation context and acquisition time. Stage 2 must re-evaluate it rather than reuse a stage-1 answer. After a failed acquisition, no cached negative is reused silently: the result is *stale* or *unavailable*, and both are distinct from undetermined.

4.5 Evidence integrity

Registry observations are hash-chained, $H_n = \mathrm{SHA256}(\mathrm{canon}(o_n) \| H_{n-1})$, and broken chains are never diffed against. A change to acquisition semantics bumps a parser version and forces a re-baseline. First detections are earliest-witness ledger entries whose content hash is verified on read. The hashes are unkeyed. They detect accidental and partial corruption, not a writer who rewrites the suffix, rolls back, or backdates a fact. Rollback and backdating resistance need witnesses outside the operator's control, such as a monotonic counter or the chain tip anchored in an external log [31]. A keyed MAC alone would not prevent replay of an older store. The guarantee is therefore *integrity against accidental and partial corruption only.* It establishes neither authenticity (who wrote the evidence), nor freshness, nor rollback resistance, and a privileged local operator, or a compromised host, is outside the adversary it resists. We call the stores hash-chained rather than tamper-evident for this reason.

5 Q2b: Applying Evidence — Applicability Specification

5.1 What a verdict means

Four predicates are kept apart: (a) *object equality*; (b) *scope membership*; (c) *availability*, meaning that a reference resolves under stated conditions; and (d) *consequence* for the dependent system. *Affected*(d, e) means (b). NotAffected(d, e) means only that d is outside e's *directly named* scope. It says nothing about availability, maintenance or vulnerability, and nothing about other events. Table 7 states what each verdict asserts and what is required before it may be issued. Every condition in its last column is checked in code and exercised by the build gate (Section 6).

Whether an event reaches a downstream distribution or an embedded component must come from the event's definition, not from identity modelling alone. An upstream end-of-life statement may concern the upstream's maintenance obligations, not a vendor's support of a repackaged image. A "contains" relation would let OpenPulse ask the question, but containment should not propagate every event automatically. Each relation needs event-specific rules. A negative verdict should therefore carry three distinct statements: outside the directly named scope; embedded-component relation not evaluated; and operational or security consequence not evaluated. Agents in particular may read a bare NotAffected label as permission to proceed. Contract 1.2.0 carries the three statements as `verdict_meaning`.

The same boundary applies to positives. AffectsArtifact means scope membership, not that a reference is broken, unsafe, vulnerable or unmaintained. Case 10 (Table 27) shows that a tag listing can establish a distribution change without establishing whether a particular reference still resolves. The contract should therefore carry four separate fields, each with its own evidence or an explicit *not evaluated*: direct scope membership; observed availability or resolution; maintenance or support implication; and security consequence, only where it is independently established. That would stop a downstream policy from turning a correct membership result into an unsupported security conclusion. Contract 1.2.0 carries them as `verdict_meaning`: direct scope, with embedded components, availability, maintenance and security consequence marked *not evaluated.* Evidence-bearing values for the last four are not yet produced.

Verdict	Asserts	Required before it may be issued
AFFECTS*	d is in the event's scope	Membership in an *exact* element, after canonicalisation; for versions, a comparison under the range's own scheme
AFFECTS-PROJECT	d belongs to the event's project; scope membership not established	Identity resolved to the project; no exact member, nothing undetermined
RELATED	An association exists but is not established	Any element undetermined: envelope membership, unresolved identity, incomparable version, mixed pinning, unknown tag, or a negative withheld (partial scope, missing basis, stale or contested event)
NOTAFFECTED	d is outside the directly named scope	N1–N5: every element excludes; identity exclusion only through a curated mapping; complete scope with recorded basis; fresh and uncontested event
UNKNOWN	No association	d is curated as another project against identity-defined elements, or no element relates to it; never because identity is unresolved or untrusted (D16 open)

Table 7: Verdict semantics. Across events, a dependency is affected if any event affects it, cleared only if every associated event gives NOTAFFECTED, and unresolved otherwise (Table 9).

5.2 Typed scope

An event e has a scope $S(e) = (E, \chi, \beta)$: a finite set E of typed elements, a completeness state χ, and the basis β of that state (Section 5.2, *Completeness*). Each element has one kind, and the denotation $[\![x]\!]$ of an element is the set of dependency references it covers:
project p: every reference that resolves to project p. It is always an envelope (below): it gives a contextual tie, never a positive, and is never used for exclusion.
repository r: every reference to repository r, whatever its tag or digest. Repository-wide changes such as relocation and withdrawal use this kind.
namespace g/n/∗: every repository under namespace n of registry g. Catalogue-wide changes use this kind, for example `docker.io/bitnami/*`, instead of listing example images that would wrongly be read as the complete scope.
tag r:t: references to r that carry exactly tag t. In place of a literal tag, an element may carry a tag predicate: `<version>` denotes every version-like tag (`1.25`, `7.2.4-debian-12-r3`, not `latest`), and `*` every tag. A predicate applies to a repository or a namespace, so `docker.io/bitnami/*:<version>` is every versioned image under the namespace. A reference with no tag is read as `latest`.
release line (p, ℓ)*:* versions of project p inside line ℓ (`4.2` covers `4.2.11`, not `4.20.1`).
object r@h: the content object with digest h, together with its OCI level (index, manifest or layer).
package (eco, n)*:* every reference to package n in ecosystem *eco*.
registry g: every reference served by registry g, for a registry-wide change such as a service withdrawal or an authentication change.
Each element also has a *role*. An *exact* element claims $[\![x]\!] \subseteq A_e$: everything it denotes is in scope. An *envelope* element claims only that it helps cover A_e. It may include unaffected references, so it can justify an exclusion but not a positive. Coverage and exactness are different premises (Section 5.4). One completeness declaration justifies negatives and says nothing about positives. The Bitnami withdrawal shows why. The versioned images moved, but `latest` stayed in the mainline. The namespace `docker.io/bitnami/*` is therefore a correct envelope, and an inexact one, since it contains `bitnami/redis:latest`. An exact representation pairs that envelope with an exact element for the versioned tags under the namespace, `docker.io/bitnami/*:<version>`.

A tag element alone would not remove `latest` from the namespace union. Roles follow from an element's form. Three forms are envelopes: a project, a namespace without a tag predicate, and a registry (the representation has no exact registry form). Every other form is exact, including a namespace with a predicate.

Each element also has a *purpose* $\pi(x) \in \{\text{bound}, \text{example}\}$, so $E = E_B \uplus E_X$. A *bounding* element takes part in the coverage claim: χ is a statement about E_B alone. An *example* is one of OpenPulse's named `affected_artifacts` (a repository, tag, object or source repository). Membership in it is a positive, but it bounds nothing, so where a bounding element of its form gives `N`, an example gives `?` for the same repository and `0` otherwise (Table 29). Project-wide scope is an explicit project element, so $E = \emptyset$ means an event that names nothing, not "the whole project".

Completeness. $\chi \in \{\text{verified}, \text{declared}, \text{partial}\}$. *Verified*: the acquisition and scope-construction procedure shows that E covers the affected set, under stated assumptions traceable to the exact responses and pagination state. *Declared*: a producer asserts it. *Partial*: not established. A deployment policy chooses the admissible states $\mathcal{A}$ for negatives: {verified} for high assurance, or $\{\text{verified}, \text{declared}\}$ under a labelled weaker policy. In OpenPulse completeness is opt-in. An unspecified scope is partial. Analysts declare completeness only where the scope is the event itself, such as one release line reaching end of life or one repository's distribution change. Curated events declare it explicitly. No verifier exists yet, so the implemented policy is $\mathcal{A} = \{\text{declared}\}$.

A well-formed declaration is not an independently justified one. A declaration is an assertion. Its only support is the evidence behind the event, and a wrong curated event, a misread announcement or a compromised producer can declare an incomplete scope complete. Then (P1) fails and a negative can be a false clear (Section 5.4). OpenPulse cannot distinguish a declaration from evidence for it, but it records what the declaration rests on. The specification therefore treats the basis as a first-class security input. Declared completeness splits into two bases that carry different evidence. A *definitional* scope is complete because it is the event itself: one release line reaching end of life, or one repository's distribution change. An *announced* scope is complete because a named source says so. A declaration records its source, the scope definition, an evidence reference, the observation time and the validation method, if any. The specification requires the contract to carry χ and that record next to every negative. Each scope records its basis (`definitional`, `announced`, `corroborated` or `verified`), and a declaration without one never clears. Contract 1.2.0 carries the completeness state, its basis and whether it admits a negative; the source, evidence reference and validation method are not carried (Section 7).

The admissibility condition is then machine-checkable. A negative is admissible iff every scope element is decisively excluded and the pair (χ, β) is in the consumer's admissible set $\mathcal{A}$, a set of (state, basis) pairs. Table 8 gives each state's evidence requirement, and what a consumer who admits it is trusting. Policies are combinations of rows. *High assurance* admits only verified scopes, so it issues no negatives today. *Definitional* admits verified and definitional scopes. *Implemented* also admits announced scopes, which is the implemented behaviour. The bases are recorded and told apart there, but the implementation admits every recorded basis, so a consumer cannot yet select a stricter set. The evaluation reports error rates separately per basis rather than pooling them (Appendix F).

Completeness is never derived from a listing. Could a compromised or misbehaving registry serve a partial tag listing, or a truncated pagination state, so that OpenPulse declares a scope complete that is not, and thereby manufacture a false clear? The claim we can support is narrower than "it cannot". It is a dataflow invariant we checked in the code. Completeness is declared at seven analyst sites in `analyzers/change_analyst.py` and in curated fixtures. Every analyst site builds E from the analysed subject itself: the release line, version or repository that

State (basis)	Evidence required	Consumer who admits it trusts	Supports a negative
Verified	A procedure binding E to the exact responses and pagination state	The procedure and the platform's state assertions	Yes if labelled, but no verifier checks the label
Declared (definitional)	The event's own definition; one release line or one repository	That the event is correctly typed	Yes
Declared (announced, corroborated)	The announcement, plus an independent source family stating the same extent	Two independent sources, and our reading of them	Yes
Declared (announced)	A named source's announcement, with an evidence reference	That source, and our reading of it	Yes
Partial	None	–	Never: exclusion from known members is not a clearance (D4)

Table 8: Completeness states, the evidence each requires, and what a consumer admitting it trusts. Corroboration counts source families, not names (Section 4). If two sources disagree on the extent of a scope, the scope is partial until the conflict is resolved. OpenPulse detects a conflict within one event's evidence (a *contradicts* relation), not between separate events.

was probed. None builds it from the members of an enumeration. So the *size* of a listing never enters E or χ. A manipulated listing can still change *whether* an event exists. That can cause a missed detection, or a false impact (a phantom removal, if the partiality goes undetected). And because a missing event drops out of the cross-event summary, it can change which events a summary is taken over. What we have not shown is an end-to-end argument that listing manipulation cannot alter event identity, scope construction, declaration choice or the loaded event set by some other route. The invariant is not yet a regression test. Same-host pagination, the page cap, the v2 fallback and the `truncated` flag reduce particular acquisition failures. They do not prove that a listing is complete, or that its sources are independent. A listing that is incomplete in a way the walk cannot detect remains a threat to discovery (Table 18). A future *verified* state would have to bind completeness to the exact responses and pagination state, and would inherit exactly this threat.

5.3 Membership tests

A test $t(d, x)$ returns M (member, with a match level), N (decisively excluded), ? (undetermined), P (project tie) or 0 (no association):

Project. P if d resolves to p. 0 if d resolves through a curated Verified mapping to another project; ? for a heuristic or unreviewed mapping (D16 is the open exception, Table 12). ? if d's identity is unresolved, including a reference that resolves only to itself, whatever its name: a mirror, fork or rename can be called anything. `github.com/etcd-io/etcd` against etcd is one such case.

Repository. M if d names r, whatever its tag or digest. Otherwise N.

Namespace. M if d's registry and namespace are g and n. Otherwise N.

Tag. M if d is $r{:}t$. N if d is $r{:}t'$ with $t' \neq t$, or another repository. ? if d pins a digest of r, because a tag can move.

Release line. 0 if d resolves through a curated mapping to a project other than p, and ? if its identity is unresolved or the resolution is untrusted. Otherwise compare versions: inside gives M, outside gives N, not comparable gives ?.

Object. Equal digests give M. For unequal digests the answer depends on the levels. If x is an index and d pins one of its children, then M if the event covers *all* children (or the deployment-selected platform's child), otherwise ?. If x is a manifest and d pins the parent index, the result is ?: the deployment's platform selection decides whether the event's platform is used. N only if both levels are known and neither contains the other. If either level is unknown, ?.
Package, registry. Names are compared in canonical form: lowercase, with PEP 503 folding for PyPI and Maven `groupId:artifactId` read as `groupId/artifactId`. Equal names give M. Unequal names give ?, because other spellings (npm scopes, Go module paths) are not canonicalised, or 0 if d is curated as another project. A value written as a purl decides by ecosystem and name: the same package gives M, a package of another ecosystem gives N, and another name in the same ecosystem gives ? (F1–F3, Appendix C). A registry element is an envelope: the same registry gives ?, a different registry gives N.
Envelopes and examples. Membership in an envelope gives ? (P for a project element) instead of M; non-membership is still N. A tag-restricted element gives ? for a reference that pins only a digest, since its tag is unknown. An example never gives N (Section 5.2). Named and bounding elements are evaluated together, and the strongest result wins.
Identity versus location. Identity decides only elements whose denotation is defined *through* a project: project, release line, and a bare package value (F3). For these, a curated VERIFIED mapping of d to another project gives 0, which needs no complete scope. Location elements (repository, tag, namespace, object, registry, source repository) are decided by location, never by d's project: a reference of another project that resides in an affected repository, namespace or registry is a member.
Across reference kinds. A package reference gives 0 against container-image, object and registry elements unless it resolves to the event's project (then ?, or P against a registry), since a package is never one of those. A Go module gives ? against a source-repository element, whatever its identity, because it may live in the repository under any path; another package gives 0. A Go module at or below a named repository path, directly or by the `gopkg.in` and `golang.org/x` conventions, gives M; positive only, since a vanity path may name the same repository (D-R1 in Table 25; Section 10). Any other comparison across kinds gives ?. The element-specific rules above apply only when the kinds agree. Appendix G gives the normative table of forms, roles, purposes and results, and two places where the release departs from it.

Algorithm 1 has three layers, and the executable oracle must implement all of them. Conformance of the second and third layers alone would validate a weaker kernel than the policy this report states. Layer 1 decides event admissibility. A withdrawn or superseded event is not evaluated. A stale or contested event may still associate a dependency, but it may never clear one. Layer 1 never changes a positive. Layer 2 is the per-element tests, on canonical references. Layer 3 is per-event aggregation, followed by the cross-event summary below. Layer 1 drops withdrawn and superseded events, withholds negatives from events contested by their own evidence, and applies the freshness bound (Section 4.4).

Across events. d is *affected* if any event affects it, *cleared* only if every event with an association returns NOTAFFECTED, and *unresolved* otherwise. Events returning UNKNOWN carry no association and are left out. In the specification that is safe, because 0 requires a VERIFIED resolution to another project, and an unresolved identity yields ? and therefore RELATED. The implementation used to detect unresolved identity only by name, and that was defect D10 (Table 12). With the etcd relocation and Bitnami events loaded, an unmapped `docker.io/zzz/kvstore` was summarised as NOTAFFECTED, while `docker.io/zzz/etcd` was RELATED. In that reproduction no pair verdict was NOTAFFECTED, but the user-facing consequence was the same as a false clear. A second reproduction of D10 was a pair-level false clear: under a version-scoped event, the unmapped reference was excluded on identity grounds (Appendix B). Since `7c0b9d9`, every unmapped reference yields ?; a heuristic mapping to another

Algorithm 1 Applicability of event e to dependency d

```
1:  function Applicability(d, e)
2:      if e is withdrawn or superseded then
3:          return Unknown                                  ▷ layer 1: not evaluated (N4)
4:      end if
5:      (E, χ, β) ← S(e);    T ← { t(d, x) : x ∈ E }       ▷ every element, no early return
6:      if M ∈ T then
7:          return Affects(strongest member level)
8:      end if
9:      if ? ∈ T then
10:         return Related
11:     end if
12:     if N ∈ T then
13:         if T ⊆ {N, 0} ∧ (χ, β) ∈ A ∧ e is fresh and uncontested then
14:             return NotAffected                          ▷ layer 1 for negatives (N4, N5)
15:         end if
16:         return Related          ▷ exclusion from a scope not known to be complete
17:     end if
18:     if P ∈ T then
19:         return AffectsProject
20:     end if
21:     return Unknown                                      ▷ T ⊆ {0}, including E = ∅
22: end function
```

project still drops out (D16, open).

Cross-event aggregation as a separate property. Per-event soundness does not imply a sound summary: D3 was a summary that displayed a clearance while another event was unresolved. The summary must keep three outcomes apart. *No association exists* (Unknown, from 0). *An association was not established* (Related or AffectsProject, from ? or P). *Decisively excluded* (NotAffected). Table 9 is the truth table over the set R of per-event results. The property is that the summary is *cleared* only if $R \subseteq \{\text{NotAffected}, \text{Unknown}\}$ and NotAffected $\in R$. The implementation's summary function takes a maximum over seven distinct ranks and then filters a set. Its result is therefore order-independent and idempotent: it depends only on which relationships occur, not on their order or multiplicity. There are 127 non-empty such sets. We checked every ordering of each of them (13,699 sequences), and every sequence of one to five results with repetition (19,607), against Table 9, with no mismatch (`papers/tr-2026-01/check_aggregation.py`). With the order-independence argument, this covers inputs of any length. It covers the summary step only. That an unresolved association reaches the summary as Related rather than Unknown is a separate property, which the D10 regression tests pin.

The table needs no further rows for the harder cases, because each of them is decided per event before aggregation. An event with a partial scope, unresolved identity, a contradiction about its extent, or stale evidence gives Related for d. A withdrawn or superseded event is not loaded, and contributes nothing. Two events with conflicting scopes are two per-event results, and the table combines them. The specification therefore guarantees that a negative from one event never erases uncertainty from another. A conflict between separate events is not detected (Section 11).

The summary is taken over the *loaded* event set, and that limits what "cleared" can mean. It states that every loaded event that associates with d excludes it. It does not state that no

Per-event results R (any number of events)	Summary
Some AFFECTS* (artifact, version or package)	affected
No AFFECTS*; some RELATED or AFFECTSPROJECT	unresolved
Only NOTAFFECTED and UNKNOWN, at least one NOTAFFECTED	cleared
Only UNKNOWN (including no events)	no association

Table 9: Cross-event aggregation. A clearance never headlines over an unresolved association, whatever the order or number of events.

relevant change exists under adequate monitoring coverage: an event that discovery missed, that a manipulated listing suppressed, or whose acquisition failed is simply absent, and the table cannot see an absence (Table 19, row 7). Neither the summary label nor contract 1.2.0 records the loaded set's coverage. A consumer must read a clearance as "excluded by the events we have", and a coverage field that says which sources and event classes were current at evaluation time is needed before a summary can support more.

5.4 Soundness of negative verdicts

Let A_e be the set of dependency references that are truly in scope of the upstream change described by e, at the time of evaluation.

Lemma 1 (Aggregation)**.** *Assume (P1)* coverage*: if* $(\chi, \beta) \in \mathcal{A}$ *then* $A_e \subseteq \bigcup_{x \in E_B} [\![x]\!]$*; and (P2)* test soundness*:* $t(d, x) \in \{\mathsf{N}, \mathsf{0}\} \Rightarrow d \notin [\![x]\!]$*. If Algorithm 1 returns* NOTAFFECTED *for* d*, then* $d \notin A_e$*.*

Proof. NOTAFFECTED is returned only when $(\chi, \beta) \in \mathcal{A}$ and $t(d, x) \in \{\mathsf{N}, \mathsf{0}\}$ for every $x \in E$. By (P2), $d \notin [\![x]\!]$ for every x. Hence $d \notin \bigcup_{x \in E_B} [\![x]\!] \supseteq A_e$ by (P1), since $E_B \subseteq E$. □

P2 for an identity-based $\mathsf{0}$ needs a correct curated mapping, and holds only because identity decides identity-defined elements alone (Section 5.3). For a location element, a correct mapping to another project does not imply $d \notin [\![x]\!]$.

Coverage makes $\bigcup_x [\![x]\!]$ an over-approximation of A_e, so it justifies negatives only. Positives need a separate premise.

Lemma 2 (Positives)**.** *Assume (P3)* exactness*:* $[\![x]\!] \subseteq A_e$ *for every exact element* x*; and (P4)* membership soundness*:* $t(d, x) = \mathsf{M} \Rightarrow d \in [\![x]\!]$*, where* $t(d, x) = \mathsf{M}$ *only for exact* x*. If Algorithm 1 returns* AFFECTS *for* d*, then* $d \in A_e$*.*

Proof. AFFECTS is returned only when $t(d, x) = \mathsf{M}$ for some element x, which is exact. Then $d \in [\![x]\!] \subseteq A_e$ by (P4) and (P3). □

Neither lemma needs the other's premises. A system can satisfy the negative premises and violate the positive ones. Before roles were implemented, OPENPULSE violated (P3): D7 and the Bitnami `latest` case were false impacts. Envelope membership no longer yields a positive, so (P3) rests on the exact elements being exact. That is a property of the event's curation, which nothing verifies. Satisfying the positive premises would not make the negatives sound. Its negatives are sound only where the negative-admission conditions hold, and they are enforced only with the gaps stated in Section 5.4 (*The* NOTAFFECTED *contract*).

The lemma is elementary, and that is intended. It is a compact correctness argument for the aggregation rule, not the primary evidence of OPENPULSE's security, and it implies no more assurance than its premises carry. The central technical evidence is meant to be conformance (Section 6), which has been run on a generated matrix and on random cases. The research question is not the implication. It is how an implementation establishes (P1) and (P2), and Table 10 decomposes them. (P1) requires a faithful event, the right elements, completeness and

currency. (P2) requires correct identity and correct version, tag and object semantics. Rows marked "not established" are the paths by which a false clear can still occur.

Two counterexamples. Both premises have failed in practice, in our own implementation. *(P1) violated:* the Bitnami event's scope listed five example images, and it was read as complete. `docker.io/bitnami/nginx` is in A_e, every element excluded it, and the verdict was NotAffected (D4). *(P2) violated:* a Bitnami-packaged `postgresql` image was resolved by a namespace heuristic to a project other than PostgreSQL, so the release-line test returned N without an established identity. If the image is a build of the release line its tag names, it is in the element's denotation, and the NotAffected verdict was a false clear (D5). In both cases the aggregation was correct, and the error came from a premise.

End-to-end statement. The property the deployed system can claim is: if OpenPulse returns NotAffected(d, e) at time τ, then $d \notin A_e(\tau)$, *provided* that every P1 and P2 row of Table 10 holds for (d, e); P3 and P4 concern positives only. The inventory premise is outside OpenPulse, which treats the inventory as untrusted input. The claim is no stronger than its weakest row, and the rows marked not established are why this report makes no unconditional claim about negatives.

The NotAffected contract. Collected in one place, a negative verdict for (d, e) is admissible only if all of the following hold: (N1) every element of E gives N or 0, after canonicalising both references; (N2) any exclusion on identity grounds rests on a curated mapping of d to a different project, never on a name or a self-resolution; (N3) $(\chi, \beta) \in \mathcal{A}$, with the declaration's basis recorded; (N4) e is current: every item the negative depends on (scope-defining evidence, the completeness basis and any identity mapping used) was acquired no later than the evaluation time and is fresh under the bound for its class and role (Section 4.4), and e is not corrected, withdrawn or superseded; and (N5) no unresolved contradiction exists about the extent of E. Otherwise the verdict is Related or Unknown, never NotAffected. Each condition is a check on recorded data, so the contract is machine-checkable once the data are recorded. (N1) and (N2) are enforced and regression-tested. (N3) is enforced: a declaration without a recorded basis never clears. (N4) is enforced for withdrawn and superseded events, and for freshness under the 90-day default of the command line and agent tools; only the library call and an explicit replay omit it. The enforced form is weaker than (N4): one bound for all classes, applied to the event's newest item, with no check that the item predates the evaluation time, and not applied to advisory-path negatives (Section 4.4). (N5) is enforced for a contradiction within an event's own evidence, in every interface. Between separate events no contradiction is detected, but the cross-event summary withholds a negative whenever any loaded event associates the dependency, so two disagreeing events yield an association, never a clear. The inputs that remain trusted are listed in Table 19. Each enforced part has a regression test. Failure to establish identity is not evidence of non-identity, and (N2) is that principle stated as a check.

5.5 Identity relations

Table 11 lists the relations that applicability needs. Each mapping carries a trust status. Curated mappings may carry a source, a review date and a maintainer, and they change through code review. They have no temporal validity and no revocation, so an outdated mapping can keep supporting negatives until someone edits it. The specification makes a mapping auditable. It carries provenance, a review time, a revalidation interval after which it degrades to Review-Required, and an explicit revocation that invalidates every verdict that cited it. A negative that rests on a mapping names that mapping in its contract. None of this is implemented.

Premise component	Established by	At the release	Checked or trusted
P1: event is faithful	Evidence gate, authority	Unsupported claims vetoed; a well-sourced wrong event is not detected	Trusted
P1: elements are the right ones	Gate (artefact classes)	Missing element not detected	Trusted; D1
P1: completeness	χ	Opt-in declaration; never verified	Trusted per basis; D1
P1: currency, non-supersession	Layer 1 of Algorithm 1	Withdrawn and superseded events dropped; 90-day bound by default in the CLI and agent tools, opt-in in the library call (Section 4.4)	Checked locally (one bound, newest item); upstream currency trusted
P2: identity	Resolver, trust status	Heuristics cannot exclude; a wrong VERIFIED mapping is not detected; a heuristic or unreviewed mapping can still remove an association (D16, open)	Curated mapping trusted; D2
P2: version semantics	Comparator, range evaluator	Scheme per range; differential for PEP 440, npm, Go and Maven; others untested	Checked (differential)
P2: tag and object semantics	Matcher	Tag elements and predicates; unequal digests withheld as undetermined	Checked (oracle)
P2: inventory fidelity	Consumer	Outside OPENPULSE: the declared reference is assumed to be what is deployed	Outside
P3: exactness (positives)	Element roles	Envelopes never give positives; exactness of exact elements rests on curation	Trusted
P4: membership soundness	Matcher, canonical form	Docker Hub hosts folded (D9); package names folded for PyPI and Maven only	Checked (oracle)

Table 10: Decomposition of the lemmas' premises. The last column says whether a premise is checked by code or tests, or is a trusted input, and which held-out measure (D1, D2; Appendix F) estimates how often a trusted input is wrong.

Policy for unresolved identity. Identity is a typed, time-bounded relation with five states: exact repository identity, verified alias, curated relationship (such as a downstream distribution), unresolved reference, and verified distinct project. Only the last supports an identity-based exclusion. The principle is that such an exclusion needs *positive* evidence that d is a different project, namely a curated, reviewed mapping. Absence of evidence is never enough: neither a dissimilar name, nor absence from alias lists, nor a heuristic rule. Weaker evidence may establish an *association*, with the confidence cap of Table 6, but never an exclusion. The last column of Table 11 applies the principle to each relation. Mirrors, forks, renames, successors and vendor-repackaged images that have no curated mapping are unresolved, and both the specification and the implementation make them ?. A package-to-repository mapping is a cross-kind comparison and gives ? unless a curated mapping relates the two (a Go module reaches the repository it lives in, positively only; Section 5.3). Whether an upstream event reaches a downstream distribution comes from the event (Section 5.1), not from the identity relation. The price is coverage. Every dependency the catalog does not know is now RELATED to events it was previously UNKNOWN or NOTAFFECTED for. The way to win clearances back is curation, not a looser rule. The OpenTofu fork, for example, is curated as its own project, so Terraform end-of-life events exclude it on that evidence. How much coverage the rule costs is a measurement in Appendix F (W1–W3).

Namespace reuse. A relocated repository and a new repository created at the old path look the same to a path-based identity. GitHub assigns every repository a stable numeric identifier, and the design records it with every observation, on any platform that exposes one. If the identifier at a path changes, that is an identity discontinuity. It becomes an event of its own, and

Relation	Representation	May exclude?
Same repository location	Equality of (registry, namespace, name)	Yes, by location
Same content object	Equal digests; OCI level not recorded	Equality only; inequality `?` (interim)
Canonical alias	Curated alias to a project; heuristic namespace rule (REVIEW-REQUIRED)	Curated: yes. Heuristic: no (D5)
Redirect / relocation	Observed path divergence; continuity of control not inferred	No
Downstream distribution, fork	Separate project with a free-text note; no "contains" relation	Only by curated mapping
Successor project	Not represented	No
Namespace reuse	Not detectable: repository IDs not recorded	No
Unresolved	Any unmapped, self-resolved reference: `?`, whatever its name	No

Table 11: Identity relations and whether each may support an exclusion.

evidence observed under the old identifier is not carried across to the new one. Earlier evidence remains valid for the time it was observed. This is not implemented.

5.6 Versions, ranges and coverage

Supported inputs. Dependency inventories are read from manifests and lockfiles for PyPI, npm, Maven and Gradle, Go, Cargo, Composer, NuGet and Swift; from CycloneDX and SPDX JSON documents; and from image inventory files. Registries are Docker Hub and OCI v2 registries, and source repositories are on GitHub. Identity is the canonical reference plus curated mappings. Package names are lowercased, PyPI names are folded by PEP 503 and Maven coordinates are read as `groupId/artifactId`; purl values decide by ecosystem and name, and npm scopes are kept (F1–F3, Appendix C). Other spellings of one package, such as Go module paths, can still compare unequal, so a bare package element gives `?` on inequality rather than `N`, and a spelling difference never clears. A dependency declared only in an unsupported format is not inventoried at all. It receives no verdict, and COMPLETE inspection says nothing about it.

The comparator implements one scheme, *generic*, as a closed grammar: a dotted numeric core, then at most one pre-release marker (`dev`, `a` or `alpha`, `b` or `beta`, `rc` or `c`, `pre` or `preview`) with an optional number, which orders within the marker. It does not implement the ordering rules of any particular ecosystem, such as dotted SemVer pre-release identifiers, PEP 440 post and local releases, Debian revisions or Maven qualifiers. A closed grammar prevents unsupported forms from being misread. It does not show that the forms it accepts are ordered as the range's scheme orders them. The generic comparator was applied to `SEMVER` and `ECOSYSTEM` ranges alike, and under SemVer its marker ranks disagree with the lexical ordering of pre-release identifiers. That was D11 (Appendix B). Each range is now compared under its own scheme. A `SEMVER` range uses SemVer 2.0 precedence. An `ECOSYSTEM` range uses the advisory's ecosystem: the generic grammar for PyPI, which agrees with PEP 440 on the forms it accepts, and SemVer for npm, Go and crates.io. Any other or missing ecosystem compares plain dotted numbers only, and equality reached only by zero padding is undetermined, because Debian orders `1.0` below `1.0.0`. Before the closed grammar, a numeric suffix was folded into the core, which was D8. Distribution suffixes (`-r75`, `+deb12u1`, `-SNAPSHOT`), unrecognised tokens and epochs (`1:2.3`, `1!2.3`) are not comparable, and yield `?`. Purely numeric date versions such as `20240101` are compared numerically, which is correct only when both sides use the same scheme. An OSV range of type `SEMVER` or `ECOSYSTEM` is a union of intervals walked in version order. `GIT` and unknown types, and versions at or past a `limit`, yield `?`. An advisory clears a version only if every affected entry does, and withdrawn advisories assert nothing. For repository inputs, COMPLETE inspection means that every file the supported readers recognise was read at a pinned

commit, and it says nothing about unsupported formats. Under PARTIAL or FAILED inspection, an absent dependency is an inspection limit and never a negative.

6 Q2c: Checking the Implementation Against the Specification

We audited the verdict engine and the version evaluator at `a461bc9`. Table 12 lists every defect the audit found at `8a9c3a8`, and one (D16) found at the release, with its minimal input and its status. Five defects found later, by the retrospective study and its follow-up (D-R1, D-R2 and D-S1–D-S3, one of them a false clear), are reported in Section 10 and Appendix C. Fixing the defects found is engineering evidence, not a demonstrated security property: it does not show that no false-clear path remains, and the late discovery of D8 to D10 suggests that some do. Each fixed defect has a regression test that fails on `a461bc9` and passes on `8a9c3a8` (Appendix C, Table 24). The expected values of those tests were written from the specification before the fixes, which is why they failed on the implementation, so they are not derived from it. They are still chosen cases, not a conformance suite. They do not cover every combination of element kind, reference kind, completeness state and identity status; the conformance matrix below does. The audit covers the paths we examined. It does not show that no other path exists.

Regression tests encoded defects. Four golden scenarios had asserted NOTAFFECTED for Bitnami-packaged images (`postgresql`, `kubernetes`, `kafka`, `cert-manager`) against end-of-life events for the upstream release lines those images contain. Their identity rested on a namespace heuristic. Fixing D5 flipped them to RELATED. Regression stability is not a correct security oracle when the expectations themselves are wrong.

Canonicalisation is semantics. Four operations are kept apart. First, parse the full reference. Second, canonicalise its registry and repository identity (case, Docker Hub hosts, `library`). Third, preserve the tag's spelling and the digest's algorithm and value. Fourth, project the reference onto what each element kind compares: the repository for repository and namespace elements, the tag for tag elements, the digest for object elements. Over-normalising creates false membership, and under-normalising creates false exclusion. Identity uses the canonical repository form, while tag and digest are read from the original reference and keep their spelling.

Specification conformance. An executable oracle (`conformance/oracle.py`) implements Algorithm 1 and the membership tests from this report's text. It imports none of the implementation, and it takes each case's identity as ground truth, so it tests matching and decision policy, not the resolver. A generated matrix crosses 17 scopes (project-wide, named example, repository, tag, both tag predicates, object, namespace envelope and exact namespace, another repository, two release lines, two package lists and two registries), three completeness states and five admissibility states with 17 dependency references. A project-wide event has no completeness to declare, so the two project-wide scopes run only as partial: $(15 \times 3 + 2) \times 5 \times 17 = 3{,}995$ cases, not the full product of 4,335. Each is run through the oracle and through the implementation, with every disagreement classified. Table 13 gives the results. The first run found three implementation defects (D12–D14), of which D12 produced eight unsafe clears. It also found four places where the specification was wrong or incomplete, and the corrected text is the one given above. Algorithm 1's layer-1 guard had made every verdict RELATED for a contested or stale event, though N4 and N5 restrict negatives only (S1). A dependency curated as another project had been given ? by the cross-kind and package rules (S2). A package had been comparable with container-image elements (S3). And a named example had been able to exclude (S4). At the release, the matrix has 21 scopes, adding qualified and mixed package values (F3) and an uncatalogued project's package (F4), and 21 dependency references, adding packages with a stated ecosystem and a package whose name is not its project's slug (F2): $(19 \times 3 + 2) \times 5 \times 21 = 6{,}195$

	Minimal failing input	Old result	Correct	Kind	Fixed
D1	Event names digests d_1, d_2; dependency pins d_2	NOT-AFFECTED	AFFECTS-ARTIFACT	false clear	`4b92f7d`
D2	Range $[1.0, 1.2) \cup [2.0, 2.3)$; version 1.1	outside	inside	false clear	`4b92f7d`
D3	Event A clears, event B RELATED	headline NOT-AFFECTED	unresolved	clearance shown	`4b92f7d`
D4	Scope of unknown completeness; dependency not listed	NOT-AFFECTED	RELATED	false clear	`f0c3b48`
D5	Heuristic identity resolves elsewhere	NOT-AFFECTED	RELATED	false clear	`4b92f7d`
D6	Version 2.5 against `fixed: 1:1.0`	outside	undetermined	false clear	`b5cbbde`
D7	Event names `redis:7.2`; dependency `redis:latest`	AFFECTS-ARTIFACT	RELATED	false impact	`bfb1ee9`
D8	`SEMVER` range `fixed: 1.0.0`; version `1.0.0-1`	outside	inside	false clear	`9d53ce2`
D9	Namespace `docker.io/bitnami/*`, declared complete; dependency `index.docker.io/bitnami/nginx`	NOT-AFFECTED	AFFECTS-ARTIFACT	false clear	`7c0b9d9`
D10	Unmapped `docker.io/zzz/kvstore:3.5`: against an etcd 3.5 end-of-life scope; and summarised with the etcd and Bitnami events	NOT-AFFECTED; headline NOT-AFFECTED	RELATED; unresolved	false clear	`7c0b9d9`
D11	`SEMVER` range `introduced: 1.0.0-beta, fixed: 1.0.1`; version `1.0.0-dev`	outside	inside	false clear	`dabafef`
D12	`docker.io/redis:7.2` against a complete scope naming `docker.io/library/redis`	NOT-AFFECTED	AFFECTS-ARTIFACT	false clear	`784fed8`
D13	Unmapped package against a release-line event	UNKNOWN	RELATED	clearance shown	`784fed8`
D14	Digest pin under a repository element	RELATED	AFFECTS-ARTIFACT	under-positive	`784fed8`
D15	Release-line member whose tag meets a named example digest	RELATED	AFFECTS-VERSION	under-positive	`d4f7d1f`
D16	`docker.io/bitnami/postgresql:16`, mapped by heuristic to another project, against a project-wide PostgreSQL event	UNKNOWN	RELATED	clearance shown	open

Table 12: The audit's defect inventory at `8a9c3a8` (later defects: Section 10). D16 was found at the release, while checking the membership table of Appendix G against the code, and is not fixed (Appendix B). D1–D3, D8, D9: implementation failures. D4: a specification gap, since completeness could not be stated and was assumed. D5: also a test-design failure, because four goldens asserted it. D7 over-matched without clearing. D10 treated an unmapped reference's self-resolution as a verified different project. That cleared it per pair under a version scope, and dropped it from summaries. Two further unsafe paths were closed by interim rules and are listed in Table 15, not here: an index digest clearing its own platform manifests, and a self-resolved reference named like the project, which D10's fix generalises.

cases. Of these, 6,123 conform. The other 72 share one label difference: for a package of the event's own project, against a registry scope or a project-wide event, the specification gives a project tie and the implementation RELATED. Both are unresolved, and both block a clearance; none is unsafe.

A hand-designed matrix tests chosen cases, so we added three further checks. *Randomised differential testing* (`conformance/randomized.py`) generates seeded cases that combine everything at once. Each has one to three events per dependency, from two projects; each event has one to four scope elements of mixed kinds, named examples, and random completeness, basis and admissibility; and the dependency is a random image or package, of either project, of a curated third project, or unmapped. Both the per-event verdicts and the cross-event summary are compared. Its first run found one more implementation defect. D15: when a named example

left a reference undetermined, the event's scope was never evaluated, so a release-line member was only RELATED. It also found that the image path still made an identity exclusion for an image curated as another project, which needs a complete scope, where S2 gives no association. At the release, with the generator extended for F2–F4, the seed-7 run agrees on all 100,000 summaries and all 200,070 per-event verdicts. *Defect re-injection* (`conformance/mutation.py`) tests the tests. It patches the running code to re-introduce each historical defect (D1, D3, D4, D7–D15) or to disable each enforced rule (N3, N4, N5, element roles, the package-name rule, S2, F2–F4): 22 single-function mutants. Every mutant is caught, and every one without help from the regression tests: by the matrix, by the randomised cases, or, for the comparator mutants D8 and D11, by the version differential below (Table 14). D2, D5 and D6 are not re-injected, because none is a single-function change of the current code. The 22 are therefore historical-defect and rule-disabling mutants, not 22 distinct historical defects. Catching them shows that the harness is sensitive to these chosen perturbations; it does not establish broad fault coverage. Finally, a *build gate* (`tests/test_conformance_gate.py`) runs the matrix, 5,000 random cases and the version differential in the ordinary test suite, in about three seconds. It fails the build on any disagreement beyond the documented label, so conformance is enforced continuously rather than measured once. The comparator was tested differentially against `packaging`, PyPA's reference implementation of PEP 440. Over 233,289 ordered pairs, it agrees on all 224,676 it decides, and it abstains on every pair `packaging` rejects. OSV ranges for PyPI agree on all 7,110 decided cases. SemVer passes the specification's own precedence vectors, both directly and through OSV `SEMVER` ranges. CI cross-checks the comparator against the pinned `node-semver` 7.8.5, `golang.org/x/mod` v0.42.0 and `maven-artifact` 3.9.16 (`evaluation/comparators/`): all 2,862 npm and 1,892 Go ordered pairs agree, and for Maven it decides 230 pairs, all agreeing, and abstains on 2,422; there are no disagreements. The range tests cover disjoint intervals, out-of-order events, `last_affected`, `introduced: 0`, `GIT` and unknown types, `limit`, mixed entries, withdrawn advisories and epochs. For other ecosystems the comparator remains a locally tested approximation of their semantics, not a validated implementation of them. Its safety rests on returning `?` for every form it does not recognise. The matrix covers chosen dimensions, not every input. Its oracle was written by the same team, so an external re-implementation would strengthen the result. Remaining conformance work is OCI level relations against recorded registry responses, and an oracle re-implemented outside the team. Agreement of any strength shows only that the implementation follows the decision policy. Because the oracle takes identity, scope and completeness as given, it says nothing about whether the inputs that admit a negative are true of the real change: whether the event describes it faithfully, its scope covers every affected reference, its completeness declaration is justified, its identity mappings are current, and the declared inventory matches what is deployed. Those are the premises of Lemma 1, and their reliability on real events is unmeasured (Section 9).

Defect mechanisms. Appendix B describes each defect's mechanism and repair: D4 in the curated Bitnami data, the tag over-match D7, and D8 to D14; D15 is described in the randomised-testing paragraph above.

Interim conservative rules. Where the full semantics are not yet implemented, fail-closed rules ensure that the gap yields `?` rather than a clearance (Table 15). They lower the rate of negative verdicts, and that rate is a quantity to measure in Section 9, not a reason to keep an unsound path. Resolving OCI levels, which would remove the interim rule for digests, is designed in Appendix E; its cost is unmeasured.

Cost. Basic costs were measured on one machine (Windows 11, 12 logical CPUs, Python 3.14) with the production engine and store (`benchmarks/cost.py`; Table 16). Matching is linear in dependency–event pairs, at about 25 µs per pair for 100 and 1,000 dependencies (24–43 µs at

Severity of disagreement (implementation vs oracle)	First run	At the release
Matrix cases	3,995	6,195
Conforming cases	3,350	6,123
Unsafe: clears, or drops an association	310	0
False impact	250	0
Under-positive (a member left unresolved)	25	0
Over-cautious	0	0
Label (both unresolved, or both non-blocking)	60	72
Randomised, seed 7: summaries agreeing	–	100,000 / 100,000
Randomised, seed 7: per-event verdicts agreeing	–	200,070 / 200,070

Table 13: Specification conformance: the matrix and randomised differential testing. The first run used the implementation before D12–D15 and the specification before S1–S4, on the 3,995-case matrix; most of its disagreements came from S1. The second column is release v0.8.0 on the extended 6,195-case matrix.

Mutant	Re-introduces	Matrix	Random	Versions	Tests
D1	First unequal digest excludes	✓	✓		✓
D3	Clearance headlines over an unresolved event		✓		✓
D4, N3	Completeness, or its basis, not required	✓	✓		✓
D7	Tags ignored	✓	✓		✓
Roles	Envelopes give positives	✓	✓		✓
D8	Pre-release numbers ignored			✓	✓
D9	Docker Hub aliases are other registries	✓			✓
D10	Unmapped references unrelated	✓	✓		✓
D11	SemVer ranges use the generic table			✓	✓
D12	`docker.io/<name>` misparsed	✓	✓		✓
D13	Unmapped packages unrelated	✓	✓		✓
D14	Repository element misses digest pins	✓	✓		✓
D15	Named example hides the scope		✓		✓
N4	Withdrawn, superseded or stale events clear	✓	✓		✓
N5	Contested events clear	✓	✓		✓
Package	Name inequality excludes	✓	✓		✓
S2	Curated other project excluded	✓	✓		✓
F2	Project package element ignored	✓	✓		✓
F3	Qualified value matched by name alone	✓	✓		✓
F4	Self-resolved package named like the project is the project		✓		✓

Table 14: Defect re-injection at v0.8.0: 22 mutants (two N4 mutants and the D4/N3 pair shown together), all caught, and all caught without the regression tests ("Tests"). Detectors: the conformance matrix, 5,000 randomised cases, and the version differential against `packaging` and the SemVer vectors.

5,000), with a 25 MiB peak. Scope size is the expensive dimension: every element is evaluated for every dependency, with no index, at about 3 µs per element per pair. Namespace elements keep real scopes small (the Bitnami event has four). The observation store writes a full 6–7 KB snapshot per repository on every sweep, even when nothing changes, and each sweep reads and re-verifies the whole hash chain, so sweep cost is linear in history length. In 1,000 consecutive sweeps of one repository (the production store, the 51-tag probe of `benchmarks/cost.py`, no tag changes), verification took about 0.16 ms per stored snapshot, and the fastest sweep rose from 9 ms at 10 snapshots to 183 ms at 1,000 (Table 16). The cumulative cost of a monitoring series is therefore quadratic in its length. Verifying from the stored tip would remove this, at the price of not re-checking old records. Through the agent interface, a cold lookup made 2–3 HTTP requests and took 0.5–0.8 s (2.5 s for the first, with connection setup); at 1–3 requests per package, an unauthenticated GitHub client allows 20–60 cold lookups an hour. A warm lookup took 2–5 ms. Inventory-driven discovery queries every target on every run, uncached,

Rule	Effect	Commit
Epoch syntax → ?	Closes D6	`b5cbbde`
Unknown OCI level on digest inequality → ?	An index digest no longer clears its own platform manifests; no digest-based negatives until levels are resolved	`f0c3b48`
Unspecified completeness → partial	Negatives only from scopes declared complete	`f0c3b48`
Self-resolved reference named like the project → ?	Such references are not dropped as unrelated; generalised to every name by the D10 fix	`f0c3b48`

Table 15: Interim conservative rules.

Measurement	Setting	Value
Matching, by inventory size (10 events)	100 / 1,000 / 5,000 deps	0.02 / 0.25 / 1.87 s
Matching, by event count (1,000 deps)	1 / 10 / 100 events	0.04 / 0.25 / 2.66 s
Matching, by scope size (1,000 deps, 1 event)	1 / 100 / 1,000 elements	0.05 / 0.42 / 3.28 s
Peak memory	5,000 deps × 10 events	24.6 MiB
Sweep, per repository (51 tags)	median of runs 2–20	35–45 ms
Sweep vs. history length (51 tags)	10 / 100 / 1,000 snapshots	9 / 51 / 183 ms
Store growth, per repository per run	steady / one new tag per run	6.1 / 7.1 KB
Agent lookup, cold (live, not repeated)	5 packages	0.5–2.5 s, 2–3 requests
Agent lookup, warm (live, not repeated)	same packages	2–5 ms, 0 requests

Table 16: Basic cost at v0.8.0 (`benchmarks/REPORT.md`). One machine; medians of three runs for matching. History length: minimum of the last 10 sweeps, under other load, not yet in `benchmarks/REPORT.md`. Matching code is unchanged since the catalogue-map cache; the earlier slowdown and its fix are in Appendix C (Table 25).

and stops at 200 repository or image targets and 500 OSV packages, reporting the rest as skipped. These microbenchmarks do not show that an organisation-wide service is viable; sustained monitoring, caches, rate limits, deduplicated review burden and indexed matching are unmeasured (`evaluation/FURTHER_STUDIES.md`).

7 Q2d: Evidence for Consumers

Evidence contract. One JSON document is produced per (dependency, event) pair and validated by a published JSON Schema. Version 1.2.0 carries the event context, the identity with its trust status, the scope with `scope.completeness` (state, basis, `admits_negative`), the match, the assessment, evidence references with authority, the temporal roles (`null` means unknown), the unknowns, `verdict_meaning` (Section 5.1), an `evaluation_context`, a recommendation, the provenance and the inspection coverage. Its `content_hash` is SHA-256 over sorted-key, compact JSON. A document that carries its own unauthenticated hash supports only an *internal-consistency* check: it shows that a document is unchanged only if the expected hash came from a trusted channel. Signature fields are always `null`, and there is no expiry, supersession or revocation. The contract is therefore suitable for evidence exchange and human review, not for automated enforcement.

Assurance levels. Three kinds of negative must be kept apart: one the implementation admits by default, one that satisfies N1–N5 in full, and one that is still suitable for a decision when it is consumed. Table 17 shows, for each condition the specification requires of a negative, whether it is computed internally, exported in contract 1.2.0, shown to an agent, and checked when the verdict is consumed. The last column is empty: a consumer can validate a document's shape and

Required for a negative	Internal computation	Contract 1.2.0	Agent response	At consumption
N1: every element excludes	Per-element results	Match method and free-text reason only	Relationship and reason	No
N2: identity excluded only through a curated mapping	Enforced	`identity_status`, `identity_via`	`identity_status`, `resolved_via`	No
N3: complete scope with recorded basis	Enforced	`scope.completeness` (state, basis, `admits_negative`); absent in 1.1	Per cause, state and basis	No
N4: fresh evidence	90-day bound on the event's newest evidence, by default in the CLI and agent tools; opt-in in the library call; advisory negatives unbounded	`fetched_at` per source; `freshness_bound_applied`	90-day bound by default on event-scope negatives; `evaluation_context`; replay mode flagged	No
N4: not withdrawn or superseded	Such events are not loaded	No expiry, supersession or revocation of the document	–	No
N5: uncontested	Within one event only	`relation` per source	Not shown	No
Meaning of the negative	Direct scope only	`verdict_meaning`	`verdict.meaning`	–
Coverage: events loaded, paths run, sources failed	Not recorded	Not exported	Per dependency (`sources`); file check: none	No
Producer authenticity	Not implemented	Signature fields `null`	–	No

Table 17: Assurance of a negative at v0.8.0: what the specification requires, where the implementation computes it, what leaves the system, and what is checked when a verdict is consumed.

its internal hash, but nothing re-checks currency, coverage or revocation at consumption time.

Agent interface. An MCP server offers three read-only tools: status of one dependency, a check of a dependency file, and supported release lines. The server is confined to its root directory and to three hosts. Its response mixes two kinds of field. *Structured* fields come from fixed vocabularies computed by OpenPulse: relationship, affected flag, confidence, identity status, event type and counts. *Free-text* fields embed upstream-derived strings: verdict reasons, which quote references and scope values, evidence URLs, repository names and licence strings. The server returns no repository descriptions, release notes or advisory prose. Even so, references and names are attacker-chosen strings. Free-text fields are not encoded or marked as untrusted, and no injection testing has been done. A deterministic verdict does not make an LLM consumer resistant to injected text in those fields. In the decision-point model of Section 3, the server is an advisory lookup at stage 1 and an inventory check at stage 3. It provides no approval or enforcement at stage 2. Every response carries a fixed note stating that its content is evidence, not a decision, and that Unknown never means safe. Each verdict carries `verdict.meaning` from contract 1.2.0, and the note tells the agent to decide from it, so a negative states what was not evaluated. The tools have two explicit modes. *Current* mode, the default and the only one the MCP server uses, evaluates as of now with a maximum evidence age of 90 days, a placeholder rather than a calibrated value; an event-scope negative from an event whose newest evidence is older becomes Related with the reason *stale*. This establishes recent acquisition of that item, not revalidation of the event's claims (Section 4.4). *Replay* mode, available only through the Python API, applies no bound and says so in the response and the note. Every

response carries an `evaluation_context` (mode, evaluation time, bound) and, for each cause, the scope's completeness state and basis. Results should be reported separately for the two modes. The interface is still an unvalidated integration boundary, not a security mitigation. It becomes one only once the structured and untrusted parts are separated and tested against a reproducible adversarial corpus. A structural separation of decision fields from upstream text, and how it would be tested, is designed in Appendix E and not implemented.

Aggregate reporting. A verdict comes from one of two paths, event scope or advisory correlation, using sources that can fail, so a missing path can read as a clean result. Every aggregate output should name (R1) the paths that ran, (R2) the event classes and events loaded, (R3) every failed or skipped source with the dependencies it would have covered, and (R4) the UNKNOWN and RELATED counts beside the affected count. The code does not meet this today: only the agent's single-dependency response meets R3, and only the digest R4. With no events and no advisory data, the command line prints "0/n dependencies affected" and exits zero; the agent's file check lists a dependency whose lookups all failed as `unflagged`; and a verdict does not record whether the advisory path ran. The change is specified, not made.

8 Threat Model

OPENPULSE reports what upstream platforms say about themselves. When an attacker controls a legitimate upstream account, the platform truthfully reports a malicious state, and OPENPULSE records it as faithfully as a benign one. Neither a verdict nor the absence of a finding is evidence of trustworthiness.

Three properties are easy to conflate, and OPENPULSE establishes only the first. *Observed platform state*: what a platform reported, from one vantage point, at the acquisition time. *Identity continuity*: that the principals or repository behind a name are the same as before. OPENPULSE observes path divergence, not control, and does not record repository identifiers. *Publisher trust*: that the content is legitimate. That is outside OPENPULSE. Provenance frameworks such as in-toto, SLSA and Sigstore help establish claims about origin and build process, and signing binds content to a key holder. Neither establishes that the published content is benign. The interface emits no trust, safety or reputation field. Each event is typed by what was observed, and the agent note says that output is evidence, not a decision. That labelling does not *prevent* a consumer from promoting observed state into trust, and no schema field could. OPENPULSE is an advisory instrument, not a validated guardrail for autonomous adoption. Signed evidence would authenticate its producer, not establish that it is correct. Negative-meaning fields would make promotion explicit and checkable. Preventing it belongs to the consumer's policy. Table 18 lists the threats to discovery, to the local store and to downstream use; threats to a negative verdict are in Table 19.

Security claim. OPENPULSE is a conditional matcher: given faithfully constructed, current events, correct completeness declarations, correct identity relations and, for a summary, every relevant event loaded, the engine issues no NOTAFFECTED inconsistent with those inputs. That is narrower than protecting dependency decisions from misleading upstream evidence. OPENPULSE checks that a declaration exists and records its basis, not that the declared coverage is correct: a declaration that omits an affected artefact yields a false clear by design, and a wrong curated mapping can do the same. Table 19 lists every input a negative depends on, whether it is checked or trusted, and the test or held-out measure that bears on it. Of the trusted inputs, D1 and D2 (Appendix F) would estimate how often declarations and mappings are wrong; nothing estimates the others, and no measurement has been made. The tests are regression tests over constructed inputs, not an adversarial evaluation over a representative population.

Property	Threat	Status	Notes
Upstream information	Selective visibility (auth, region, rate limit)	partial	Errors and truncation recorded; vantage point not recorded
Discovery	Change during pagination	not handled	Traversal over an interval, not a snapshot
	Silently partial or manipulated listing	partial	Listing size never enters E or χ (checked dataflow invariant); can cause missed detections, phantom removals, and a changed set of summarised events
Local evidence	Accidental or partial corruption	detected	Hash chain
	Suffix rewrite, rollback, backdating	not handled	Unkeyed; no witness
Downstream use	Injection via free-text fields	not handled	Structural separation designed (Appendix E), not implemented; no testing
	Negative read as clearance	partial	Meaning carried in contract 1.2.0 and the agent response; a consumer may ignore it
	Older contract substituted for a current one	not handled	Internally consistent; unsigned, no expiry or supersession

Table 18: Threats outside the negative decision.

Input	How a wrong input, or an attacker controlling it, affects a negative	Status
Trusted: no mechanism of OpenPulse detects an error		
1 Platform-reported state	A compromised account reports a malicious state truthfully; recorded, never read as trust	Out of scope
2 Event construction	A well-sourced wrong event, or a missing element, excludes an affected reference; copies counted as independent raise its confidence	Trusted; D1 (elements)
3 Completeness declaration	Undeclared: never clears. Declared complete while omitting an artefact: false clear (pinned); basis recorded	Trusted per basis; D1
4 Curated mapping, bare-name alias	A wrong, stale or reused-namespace mapping excludes an affected reference; no expiry or revocation	Trusted; D2
5 Supersession	A wrongly narrow superseding record replaces a correct one	Trusted; D1 (its scope)
6 Upstream currency	A stale or replayed response fetched today passes the bound; source timestamps unchecked	Trusted; not measured
Trusted for summaries only, not a premise of a per-event negative		
7 Loaded event set	A missed, suppressed or failed acquisition removes an event; the summary clears over what is loaded, and a failure does not withdraw a negative	Trusted; not measured
Checked: pinned by tests on constructed inputs		
8 Names, redirects, tags, references	Exclusion needs a curated mapping; an unmapped lookalike resolves only to itself (D10), but a heuristic mapping can still drop an association (D16, open); a Go module reaches its repository positively only (D-R1); a stated ecosystem needs a curated (ecosystem, name) pair; literal tags compare by exact spelling; unequal digests and uncanonicalisable references are undetermined	Checked; D16 open
9 Local evidence age	A stale event cannot clear under the default bound (agent, CLI; not the library call), judged on its newest item; a past `-as-of` admits later evidence, and advisory negatives are unbounded (both open); retracted events dropped	Checked; two gaps open
10 Consistency	Within an event a contradiction withholds the negative (N5); across events any association beats a clear	Checked
Outside OpenPulse		
11 Inventory and consumer	The declared reference is assumed to be what is deployed; upstream text never changes a verdict, but whether an agent respects unresolved states is untested	Outside; agent untested

Table 19: Every input a negative depends on, at v0.8.0. "Trusted": a premise; no D1 or D2 result exists, and rows 1, 6 and 7 have no planned measure. "Checked": pinned by `tests/test_adversarial_inputs.py` or the regression suite on constructed inputs. A negative is no stronger than its weakest row.

9 Q3: Prospective Evaluation

Apart from conformance (Section 6), the cost measurements (Section 6) and the exploratory study of Section 10, nothing in this section has been run, and nothing in it is evidence. A registration draft fixes the endpoints, margins, guardrails, sample-size rule, stopping rule, inclusion criteria and analysis code before any held-out label exists (`evaluation/REGISTRATION.md`, `evaluation/registration.py`). Its independence requirement needs two annotators, an adjudicator and an encoder outside the author, so it is offered as an open protocol: a team that runs it archives the registration in its own name before labelling. Its main baselines are the minimal conservative reference (iv), which never clears; the strong conservative baseline (v), which shares OpenPulse's model; and the existing-representation baseline (vi), which reads the same events re-encoded as OSV and OpenVEX (Algorithm 2). Table 20 lists what it tests. Appendix F gives the metrics, annotation, baselines, population and leakage controls. Studies of discovery, agent decisions and cost at scale are designed in `evaluation/FURTHER_STUDIES.md` and are not registered.

What each comparison can show. The baselines answer different questions and are not interchangeable competitors. The naive baseline shows the cost of reading absence as exclusion. The minimal reference (iv) never clears. It therefore plays no part in the safety test, which is absolute, and beating it in resolution (H2) shows only that the model's clearing rule resolves more than a policy that reads no negatives. H2 is a minimum, not a comparison with careful practice. The practical claim is that the typed model resolves more than existing formats read with care, at equal safety. S3 tests that claim, and its result is reported next to the primary verdict whatever H2 shows.

Identity knowledge differs between baselines: (iv) matches bare names, (vi) exact purls, and (v) and OpenPulse the curated catalogue, so a chain of baselines would mix curated identity with the model, an effect the retrospective study shows is not small (Section 10). The draft therefore crosses two factors and holds the decision consumer fixed (Table 21). *Representation* is a literal affected list (L), the event re-encoded as OSV and OpenVEX (X), or typed elements with χ (T). *Identity knowledge* is exact canonical identity (0) or the closure under one frozen catalogue snapshot (K). Every cell uses the oracle's parser and the pinned comparators, and clears only where its representation licenses it. The model's contribution is $T_K - X_K$ (and $T_0 - X_0$), at fixed identity. Curated identity's contribution is $T_K - T_0$ and $X_K - X_0$, at fixed representation. Their interaction says whether the model's advantage depends on curation. The engine's contribution is OpenPulse $- T_K$ (S1). S3, OpenPulse $- X_K$, is the sum of the model and engine contributions. Every contrast is reported with its interval, the false-clear events of both cells and its review cost (W1), overall and on events with no encoding loss. Only T_K and X_0 are implemented; L_0, L_K, X_K and T_0 are specified in Algorithm 2 and not yet implemented, so the draft cannot be archived until they are.

ID	Quantity	Comparator	Rule	Role
H1	Event false-clear share: events with an in-scope pair reported NotAffected, over events at risk	none (absolute)	Exact one-sided 95% upper bound ≤ 0.05; needs ≥ 60 events at risk, else "not decidable"	primary
H2	Resolution (Eq. 3), paired	(iv)	Difference ≥ 0.10 and one-sided 95% lower bound > 0	primary
S1	Resolution, paired	(v) $= T_K$	90% interval within ± 0.05 (equivalence)	secondary
S2	Event false-clear share	naive A	A minus OpenPulse > 0	secondary
S3	Resolution, paired	X_K	One-sided 95% lower bound > 0; reported beside the primary verdict	secondary
G1	Silent misses: no association, over in-scope pairs	–	≤ 0.02	guardrail
G2	Unresolved (Related or AffectsProject), over determinable pairs	(iv)	$\leq$ (iv)'s $+ 0.05$	guardrail
G3	Unresolved rate per event class	–	Flagged above 0.60	guardrail
G4	Median W1 over inventories	(iv)	$\leq$ (iv)'s	guardrail
W1–W3	Unresolved per inventory and cycle; per event at risk; reviewer minutes per item	OpenPulse, (iv), (v), (vi)	–	reported
D1, D2	Under-inclusive completeness declarations per basis; wrong or stale identity mappings	–	Exact intervals; H1 is not reported as safety evidence without them	necessary
–	Decisive coverage (Eq. 2)	all	–	reported

Table 20: Endpoints of the registration draft. Resolution, the H2 endpoint, counts correct decisive outcomes over determinable pairs; decisive coverage counts decisive outcomes, right or wrong, over all pairs, and is never tested (Appendix F). Comparators are the cells of Table 21. Secondary tests are Holm-corrected and never rescue a failed primary result. Guardrails are reported beside the verdict and do not change it.

Representation	Exact identity ($k = 0$)	Frozen catalogue ($k = K$)
L: literal list, absence abstains	L_0	L_K
X: OSV / OpenVEX, careful consumer	X_0 = baseline (vi)	X_K (S3)
T: typed elements, χ, $\mathcal{A}$	T_0	T_K = baseline (v)
Engine	–	OpenPulse (S1 against T_K)

Table 21: The factorial comparison. Each cell runs the decision rule of Algorithm 2 for its representation, with the identity function of its column. Outside the grid: the naive policy A (S2) and the minimal reference (iv), which reads a literal list with bare-name identity, never clears, and is the comparator for H2, G2 and G4.

10 Q3: Retrospective Study on Historical Changes

During development, OpenPulse detected six non-lifecycle changes, all after they took effect (Appendix D); they show detectability, not effectiveness, and are not used as evidence here. This section reports an exploratory, retrospective study (R-C0) that applies OpenPulse to real upstream changes from 2016–2025 and to real dependency references read at commits from before each change. It is not the prospective evaluation of Section 9 and is not pooled with it. Its design, every amendment, the frozen case file and the results are in `evaluation/retrospective/` of the repository; this section summarises `REPORT.md` there.

Design. Twenty-seven events were screened into six classes under quotas fixed before any data were collected: relocation (3), deprecation and archival (6), distribution (6), licence change (4), end of life (3) and compromised package versions (5). Each event is a fixture built only from archived primary sources (Wayback snapshots or immutable records). For each event we

	In-scope pairs (103)				Rates			
Policy	Warned	Unres.	Silent	Cleared	F. impact	Resolved	Decisive	Abstained
A naive	68	0	0	35 (10)	3.2%	77.6%	100%	0.0%
B conservative	68	35	0	0 (0)	3.2%	41.2%	41.9%	58.1%
V strong conservative	61	42	0	0 (0)	1.6%	47.3%	47.3%	52.1%
C OpenPulse at the freeze	64	27	12	0 (0)	1.6%	50.3%	50.3%	38.9%
C after fixes (post hoc, diagnostic)	77	26	0	0 (0)	1.6%	58.2%	58.1%	38.3%

Table 22: Retrospective study: 167 pairs over 27 events. The first four columns partition the 103 in-scope pairs: *Warned*, decided affected; *Unres.*, associated but unresolved; *Silent*, no association; *Cleared*, a false clear (events in brackets). Only *Warned* delivers the warning. *F. impact*: out-of-scope pairs decided affected, over out-of-scope pairs. *Resolved*: correct decisive decisions over the 165 determinable pairs (resolution, the H2 endpoint, Eq. 3). *Decisive*: Affects* or NotAffected, right or wrong, over all 167 pairs (decisive coverage, Eq. 2); for C at the freeze the two coincide only after rounding (83/165 and 84/167). *Abstained*: Related or AffectsProject over all pairs, the review burden; no-association outcomes (C: 18 at the freeze, 6 after fixes) are in neither rate. A decided both undeterminable pairs; no other policy decided either. The last row is a separate run after two fixes written on the same events; it is diagnostic and is not the study's result.

collected real references that should be in scope and nearby references that should not, each a file in a public repository at a stated commit inside the event's window: 102 references collected as in-scope candidates and 65 neighbours, none synthetic, giving 167 pairs. One reference fell outside its window and was excluded; no event was dropped. Labels follow one written rule per event, taken from that event's primary source, and not the role a reference was collected in: 103 in scope, 62 out of scope and 2 undeterminable. The two differ on three neighbours, and no label was changed by adjudication. One neighbour, collected as the same name in another namespace (`github.com/golang/lint`), is golint's own former import path, so the deprecation's rule labels it in scope. Two, CircleCI images that ship Python 2.7 and Node.js 12, are downstream distributions whose support the upstream end of life does not settle, and are labelled undeterminable. The implementation was frozen at tag `retro-freeze-2026-10-30` (`3639a0f`) and the case file before any policy ran (SHA-256 recorded and checked in CI). The run used the pinned ecosystem comparators (`node-semver` 7.8.5, `golang.org/x/mod` v0.42.0, `maven-artifact` 3.9.16) and a 90-day freshness bound. Each event was replayed at the date of its newest evidence, so it is evaluated as it was knowable then, and the bound could not bite on that item. A second run reproduced the results file byte for byte.

Labels. All labels are the developer's. Two rounds of outside labelling on a seeded 25% sample (42 pairs) turned out to be AI-assisted (a language model prepared the second round), so they are reported only as a consistency check ($\kappa = 0.722$ and 0.825 against the developer's labels), never as inter-rater reliability. The pre-specified analysis, whose population is independent labels only, is therefore empty, and every test below is illustrative.

Policies. The rules are those of Algorithm 2 (Appendix F). A (naive) reports affected if any current event lists the dependency by bare name, and otherwise not affected. B is the minimal reference (iv): affected if listed, and otherwise an abstention; it never clears. V is the strong conservative baseline (v), the cell T_K, compared with C by the equivalence test S1. C is OpenPulse. Baseline (vi) is not applied here. The results on these events were known before any encoding could be written, so the encodings could not be blind.

Decisions. Table 22 gives the result at the freeze. The primary finding is descriptive and needs no sampling model. On these 27 events, under the developer's labels, A falsely cleared 35 of 103 in-scope pairs in 10 events, every in-scope pair in 8 of them, and C cleared none. Avoiding false clears is half of the result. C warned on 64 in-scope pairs, fewer than B's 68: it left 27 unresolved and missed 12 silently, so 39 received no warning. On the five compromised-package events B warned on all 16 in-scope pairs, because the event lists the package, and C on none: those packages have no curated identity, and the identity rule (Section 5.5) does not treat an uncatalogued name as the event's package. C's higher resolution comes from correct clears of out-of-scope pairs, not from more warnings. Its review burden is lower: 65 of 167 pairs unresolved, against B's 97 and V's 87.

Inferential model. The bound and the test below hold only if the 27 events are independent draws from a fixed population of upstream changes, with one per-event false-clear probability per policy, and the labels are correct. The design meets none of these conditions (Section 11). Under that model, C's one-sided 95% upper bound per event is 0.105 ($1 - 0.05^{1/27}$). B, which never clears, has the same bound, so the figure reflects the event count, not C, and is not a bound on a deployed rate. It is above the margin, and 27 is below the 60 events the safety test needs, so non-inferiority is not decidable. Under the same model, S2 tests A's excess per-event false-clear share over C's ($10/27 = 0.370$). Its event-resampled bootstrap is at the floor of 4,000 resamples: Holm-adjusted $p = 0.0005$, the smallest value it can return (`REPORT.md` gives 0.0004, adjusted after rounding); an exact sign test on the 10 discordant events gives 0.002. The difference is thus not a resampling accident on these events; the test cannot correct for how they were chosen or who labelled them. A pair-level bound (0.029) would treat clustered pairs as independent and is not reported. For A, 35 of 103 in-scope pairs (0.340; event-clustered upper bound 0.486) and 10 of 27 events (upper bound 0.547).

Resolution. The gain in resolution over B (0.091, 95% CI $[-0.054, 0.245]$) misses the pre-specified 0.10, and equivalence with V within ± 0.05 is not shown (0.030, 90% CI $[0.000, 0.075]$). The gain over B mixes representation and identity. It is 15/165: C's 19 correct clears, which B cannot make, plus 16 in-scope positives C reached through typed release-line and package scopes, minus 20 that B reached and C left unresolved. Of those 20, 16 are compromised packages that the catalogue does not contain, which B matched by bare name, and 4 are references under a registry-wide scope that B read as a listing and C treats as an envelope. The comparison with (iv) is therefore not a measure of the model. The factorial of Table 21 separates the two factors; its cells that need no encoding (L_0, L_K, T_0, T_K) could be run on these events post hoc, and would be diagnostic only. C's advantage is concentrated in end of life (88.2% resolved against V's 58.8%).

A defect the study exposed. Guardrail G1 (silent miss ≤ 0.02) fails: 12 of 103 in-scope pairs (11.7%) received no association. Every one is a Go module reference, such as `github.com/docker/docker`, to a repository the event names as a source-repository artefact. The engine never compared a Go module path with a repository artefact, although a Go module path is a repository locator. V left all twelve unresolved: not a decisive positive, but each would reach a reviewer, where C gave no association. A silent miss is not a false clear, since no negative is issued, but it withholds the warning the event exists to give. The dependency shows as UNKNOWN, and no command-line threshold fails on UNKNOWN or RELATED, so a CI gate on affected dependencies would have passed all 39 unwarned in-scope pairs (Section 7).

Where C did not resolve. C resolved 83 of the 165 determinable pairs. Of the other 82, 17 involve the Go defect: the 12 in-scope silent misses, and 5 out-of-scope Go module neighbours (the new location or a different project) that also received no association. Those

5 are neither false clears nor false impacts, but they count against resolution, and the fix, positive only, leaves them unchanged. A further 44 come from packages with no catalogued identity (all 31 compromised-package pairs among them), and 20 from representation gaps (19 unresolved and the one false impact). The remaining one is a non-association with a different project (OpenTofu), right in substance but not a decisive clear. The representation gaps are scope forms the specification cannot yet state exactly: registry-wide scopes (only as envelopes), open-ended version boundaries ("every release from 1.6.0"), a date boundary across release branches (MongoDB's SSPL change), and exclusions (Early Access tags). The fixtures state each as an over-approximation of the announced scope. That costs resolution and can cause a false impact, as the Early Access tag did, but it cannot cause a false clear: it preserves negative soundness, not precision.

Discovery. For the 15 relocation, archival and distribution events, OpenPulse's live collectors detected none as shipped, because collection was keyed by catalogue entry and the projects were not catalogued. Given each repository or image directly, they detected 5, all archivals, each with a project-wide scope broader than the announced one. They missed every relocation: the collector recorded GitHub's 301 redirect for a transferred repository as an error, so the ownership comparison of Appendix D never ran. Changes stated only in a notice, registries other than Docker Hub, and tag deletions visible only across observations are outside what the collectors observe.

After fixes. Both defects were fixed in one change (Appendix C): a Go module now reaches the repository it lives in, positively only, and the collector follows one redirect. The frozen case file was then run again on the freeze tag plus those two fixes and nothing else. Thirteen decisions changed, all to correct: the 12 silent misses, and golint's former path `github.com/golang/lint`, which was unresolved rather than silent and is counted above among the uncatalogued packages; G1 passes; the gain over B becomes 0.170 (one-sided lower bound 0.031); C leads V by 0.109; there is still no false clear; and discovery detects all three relocations, naming exactly the announced artefact. These figures are post hoc and diagnostic: the fixes were written after seeing the errors on the same events, so they show what caused the errors, not unbiased performance. They are never pooled with, or cited in place of, the freeze-tag result.

Where C and V differ. After the fixes, C and V decide 23 of the 167 pairs differently, only through the two mechanisms named for (v) in Appendix F: 18 Go module references to a relocated or deprecated repository, which V leaves unresolved and C decides (the 13 in scope affected, the 5 neighbours with no association), and 5 image tags with a variant suffix (`node:12-alpine`, `node:16-alpine`, `python:3.7-slim`), which V cannot compare and C decides correctly, three affected and two cleared. Every difference favours C under the developer's labels, and none involves completeness, identity or admission, which the two share. The study therefore gives no evidence that OpenPulse's value lies in its engine rather than in the shared model; at the freeze, before the Go fix, the two differed on 22 pairs and C's lead was 0.030.

What the completeness condition changed. Twenty of the 27 fixtures declare a complete scope, six are partial and one has none; none mixes exact and envelope elements. Only three in-scope pairs lie outside a stated partial list (Elasticsearch 7.12–7.13 against 7.11), and the identity rules decide them first. Assuming every scope complete changed seven decisions, all out-of-scope neighbours that the ablation cleared correctly (resolution 50.3% to 54.5%; the same seven after the fixes). The condition thus withheld seven correct negatives and prevented no false clear. Freshness and contradiction rules were not exercised: replay keeps the newest evidence fresh, and no evidence contradicts another.

A false clear outside the study's measurement. The study's C applies events only; it passes no advisory data, so OpenPulse's second decision path, which correlates OSV and NVD advisories by package and version, never ran. Closing the compromised-package gap by querying OSV per package then exposed a defect in that path that the audit of Section 6 had not found. Advisories were merged by CVE identifier, and an advisory without one was dropped. Most advisories for malicious or sabotaged packages have none, so `node-ipc` 9.2.2, the version that shipped the protestware dependency, was reported NotAffected against an unrelated CVE. Two false-impact defects in the same reader were found alongside it (D-S1–D-S3 in Appendix C). All three are fixed, with regression tests that fail on the old code. The study's safety result therefore covers one decision path, event scope, and is not a claim about the product. The advisory benchmark is designed, not run (Appendix F).

11 Limitations

Evidence of value. No independent effectiveness result exists, and confidence levels are uncalibrated. The development ledger shows detectability only (Appendix D). The retrospective study has a denominator but not independence: every label and fixture is the developer's, the events were selected under quotas, and their dependents were found in repositories that still exist. Its inclusion criteria (archived sources, machine-identifiable identifiers, at least three findable public dependents) favour events whose scope can be written down and whose dependents are visible, perhaps those a scope-based engine handles best. Its bound and test assume a sampling model the design does not satisfy, and its 27 events cannot decide the safety test, which needs 60. It measured one decision path, event scope; the advisory benchmark is designed, not run. At the freeze 39 of 103 in-scope references received no warning (Section 10). Of the factorial cells only T_K and X_0 exist, and (vi) is only as strong as its encoding rules and encoder.

Trusted inputs and soundness premises. The lemmas are conditional, and several premises are not established (Table 10). Completeness is declared, never verified, though each declaration records its basis. Whether declarations and curated mappings are correct is the principal unmeasured determinant of negative soundness; event faithfulness, supersession, upstream currency and the loaded event set are trusted as well (Table 19). None is measured. The regression tests are chosen cases, and the conformance oracle was written by the same team.

Admission gaps. The freshness bound is the default in the agent tools and the command line, not in the library call. It is one 90-day bound on an event's newest evidence, not a per-class bound on each item a negative depends on; it establishes recent acquisition of that item, not revalidation of the others or validity when the verdict is consumed. A past `-as-of` admits evidence acquired later, and advisory negatives are never bounded (Section 4.4). Conflicts between separate events are not detected, and a failed acquisition does not withdraw a negative. Interim rules withhold negatives where semantics are missing (OCI levels). Package names are folded for PyPI and Maven only; element roles are not labelled in the contract; version orderings other than SemVer, PEP 440 and plain numbers are undetermined.

Defect history. Fifteen of the sixteen defects are fixed; D16 is open, and so is a latent gap between packages and source repositories (Appendix B). Seven of the thirteen defects that could produce or display false clears were found after the initial audit, five by external questions and two by the conformance matrix, and the retrospective study then exposed two more, so further defects are likely. Closing D10 makes every unmapped dependency Related, which caused 44 of C's 82 non-correct decisions in the study; its review cost per inventory and cycle is unmeasured. A policy that always abstains has no false clears, so the question is whether OpenPulse makes more justified decisions than (v) at an acceptable review burden.

Scope, consumption and scale. Discovery covers three classes on two platforms and English-language sources, cannot see private registries, and derives targets only from Go module paths, `github.com` references and Docker Hub images. No advance warning has been demonstrated for an unrecorded class. Local evidence and contracts are unauthenticated, and first detections can be backdated. The agent interface is an unvalidated integration boundary, untested against injection. No aggregate output reports the paths, classes and failed sources of a run (Section 7). Matching has no index, acquisition no backoff or scheduling, discovery is capped per run, and sweep cost grows linearly with history (Section 6). Costs were measured on one machine, not under sustained monitoring.

12 Conclusion

OpenPulse collects evidence about upstream changes that vulnerability and lifecycle records do not carry, qualifies it, and applies it to inventories under a specification in which a negative requires a complete scope. Our audit of the implementation found sixteen defects, thirteen of them ways in which a false clear could arise or be displayed, some of which the regression suite had treated as correct; fifteen are fixed and one is open. An executable oracle and differential testing found further defects and four specification errors, which is why conformance must come before claims of effectiveness. A first retrospective study on 27 selected changes, labelled by the developer, found no false clear by the event-scope engine where a naive reader issued 35 in 10 events, through typed membership and identity, not the completeness condition, which no event exercised. The engine also left 39 of 103 affected references without a warning, 12 of them silently. That describes those events, not a population rate. The study also found a reference form the engine never matched, a blind discovery path and a false clear in the advisory path; the figures re-measured after the fixes are diagnostic. The defensible thesis is therefore narrow: a testable assurance model that makes the conditions for a negative explicit, an audit of one implementation against it, and a pre-specified, not yet run study of its safety–coverage trade-off against a strong conservative baseline and a careful consumer of existing formats, offered for independent teams to run. The assurance is conditional and covers one decision path: given faithfully constructed, current events, correct completeness declarations, correct identity relations and, for a summary, every relevant event loaded, the engine issues no negative inconsistent with those inputs; whether they deserve that trust is unmeasured. Whether the evidence changes dependency decisions, for developers or for agents, is the question that study and a separate agent experiment are designed to answer.

Availability and Collaboration

OpenPulse is open source under the Apache License 2.0 at `https://github.com/IkarusAILabs/OpenPulse`. It includes its schemas, recorded fixtures and replay tests. It is developed by Ikarus AI Labs, a non-profit research initiative based in Cambridge, UK, as part of a programme on evidence-based assurance for AI-mediated software change. We welcome collaboration, in particular from researchers and practitioners independent of the detector's development. Useful roles are independent annotation and reference-set curation, contributing real inventories that contain distributions and embedded components, running the agent experiment, and integrating the evidence contract into dependency-decision workflows and assurance tooling. See `https://ikaruslabs.net/`.

AI Assistance and Authorship

This report has a single author, who is responsible for all of its content. AI coding and writing assistants, including Anthropic's Claude through Claude Code, were used throughout:

- for the implementation, the tests, the evaluation harness and its analysis code;
- for drafting and revising the text, including the responses to external reviews.

Commits made with that assistance name it in the repository history. The author directed the work, made every design and methodological decision, and reviewed what the assistants produced. None of the report's data are independent of the author:

- the development set and the retrospective study were labelled by the author;
- the outside labelling round was itself AI-assisted, so it is reported only as a consistency check (Section 10);
- the development-set encodings for baseline (vi) are the author's.

This is why the report makes no confirmatory claim, and why its prospective protocol requires annotators independent of the author.

References

[1] Afsah Anwar, Ahmed Abusnaina, Songqing Chen, Frank Li, and David Mohaisen. Cleaning the NVD: Comprehensive quality assessment, improvements, and analyses. *IEEE Transactions on Dependable and Secure Computing*, 19(6):4255–4269, 2022.

[2] Guilherme Avelino, Eleni Constantinou, Marco Tulio Valente, and Alexander Serebrenik. On the abandonment and survival of open source projects: An empirical investigation. In *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–12. IEEE, 2019.

[3] Bitnami. Upcoming changes to the Bitnami catalog (effective August 28th, 2025). GitHub issue bitnami/containers#83267, July 2025. Opened July 16, 2025. Accessed 2026-10-28.

[4] Bodin Chinthanet, Raula Gaikovina Kula, Shane McIntosh, Takashi Ishio, Akinori Ihara, and Kenichi Matsumoto. Lags in the release, adoption, and propagation of npm vulnerability fixes. *Empirical Software Engineering*, 26(3), 2021.

[5] Jürgen Cito, Gerald Schermann, John Erik Wittern, Philipp Leitner, Sali Zumberi, and Harald C. Gall. An empirical analysis of the Docker container ecosystem on GitHub. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, pages 323–333. IEEE, 2017.

[6] E. F. Codd. Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4):397–434, 1979.

[7] Filipe R. Cogo, Gustavo A. Oliva, and Ahmed E. Hassan. Deprecation of packages and releases in software ecosystems: A case study on npm. *IEEE Transactions on Software Engineering*, 48(7):2208–2223, 2022.

[8] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252. ACM, 1977.

[9] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *18th USENIX Security Symposium*. USENIX Association, 2009.

[10] Cybersecurity and Infrastructure Security Agency. Known exploited vulnerabilities catalog. CISA. Accessed 2026-10-28.

[11] Cybersecurity and Infrastructure Security Agency. Minimum requirements for vulnerability exploitability eXchange (VEX). Technical report, CISA, April 2023. Published April 21, 2023.

[12] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, 2024.

[13] Alexandre Decan, Tom Mens, and Philippe Grosjean. An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1):381–416, 2019.

[14] Ecma International. ECMA-424: CycloneDX bill of materials specification. Standard, Ecma International, December 2025.

[15] endoflife.date contributors. endoflife.date. Website and API. Accessed 2026-10-28.

[16] Oren Etzioni, Keith Golden, and Daniel S. Weld. Sound and efficient closed-world reasoning for planning. *Artificial Intelligence*, 89(1–2):113–148, 1997.

[17] GitHub. GitHub advisory database. GitHub repository github/advisory-database. Advisories stored in OSV format. Accessed 2026-10-28.

[18] GitHub. Transferring a repository. GitHub Docs. Accessed 2026-10-28.

[19] Ilay Goldman and Yakir Kadkoda. GitHub dataset research reveals millions potentially vulnerable to RepoJacking. Aqua Security (Aqua Nautilus) blog, June 2023. Published June 21, 2023. Accessed 2026-10-28.

[20] Google. Open source insights (deps.dev). Service and API documentation. Accessed 2026-10-28.

[21] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90. ACM, 2023.

[22] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, Huajun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. Investigating package related security threats in software registries. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1578–1595. IEEE, 2023.

[23] Tomasz Imieliński and Witold Lipski. Incomplete information in relational databases. *Journal of the ACM*, 31(4):761–791, 1984.

[24] Nasif Imtiaz, Seaver Thorn, and Laurie Williams. A comparative study of vulnerability reporting by software composition analysis tools. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–11. ACM, 2021.

[25] Elgun Jabrayilzade, Mikhail Evtikhiev, Eray Tüzün, and Vladimir Kovalenko. Bus factor in practice. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 97–106. ACM, 2022.

[26] Stephen Cole Kleene. *Introduction to Metamathematics*. North-Holland, 1952.

[27] Arjun Krishna, Erick Galinkin, Leon Derczynski, and Jeffrey Martin. Importing phantoms: Measuring LLM package hallucination vulnerabilities, 2025.

[28] Raula Gaikovina Kula, Daniel M. German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. Do developers update their library dependencies? *Empirical Software Engineering*, 23(1):384–417, 2018.

[29] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. SoK: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1509–1526. IEEE, 2023.

[30] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, RFC Editor, June 2013. Experimental; obsoleted by RFC 9162.

[31] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, RFC Editor, December 2021. Experimental.

[32] Alon Y. Levy. Obtaining complete answers from incomplete databases. In *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*, pages 402–412, 1996.

[33] Frank Li and Vern Paxson. A large-scale empirical study of security patches. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2201–2215. ACM, 2017.

[34] Changyuan Lin, Sarah Nadi, and Hamzeh Khazaei. A large-scale data set and an empirical study of Docker images hosted on Docker Hub. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 371–381. IEEE, 2020.

[35] Courtney Miller, Christian Kästner, and Bogdan Vasilescu. "We feel like we're winging it:" a study on navigating open-source dependency abandonment. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1281–1293. ACM, 2023.

[36] Amihai Motro. Integrity = validity + completeness. *ACM Transactions on Database Systems*, 14(4):480–502, 1989.

[37] National Institute of Standards and Technology. National vulnerability database (NVD): Program updates. NIST. Status notices on the CVE enrichment backlog since early 2024, incl. updates of March 19, 2025 and April 15, 2026. Accessed 2026-10-28.

[38] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367. ACM, 2022.

[39] OASIS. Common security advisory framework version 2.0. Oasis standard, OASIS Open, November 2022.

[40] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber's knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)*, Lecture Notes in Computer Science, pages 23–43. Springer, 2020.

[41] OpenSSF. Open source vulnerability (OSV) format. Schema specification, version 1.9.2. Original authors Oliver Chang and Russ Cox. Accessed 2026-10-28.

[42] OpenVEX Project. OpenVEX specification. GitHub repository openvex/spec. Draft; JSON-LD context v0.2.0. Accessed 2026-10-28.

[43] Ivan Pashchenko, Henrik Plate, Serena Elisa Ponta, Antonino Sabetta, and Fabio Massacci. Vulnerable open source dependencies: Counting those that matter. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–10. ACM, 2018.

[44] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 754–768. IEEE, 2022.

[45] Henrik Plate, Serena Elisa Ponta, and Antonino Sabetta. Impact assessment for vulnerabilities in open-source software libraries. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 411–420, 2015.

[46] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 449–460. IEEE, 2018.

[47] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. Detection, assessment and mitigation of vulnerabilities in open source dependencies. *Empirical Software Engineering*, 25(5):3175–3215, 2020.

[48] Simon Razniewski and Werner Nutt. Completeness of queries over incomplete databases. *Proceedings of the VLDB Endowment*, 4(11):749–760, 2011.

[49] Raymond Reiter. On closed world data bases. In *Logic and Data Bases*, pages 55–76. Springer, 1978.

[50] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999.

[51] Rui Shu, Xiaohui Gu, and William Enck. A study of security vulnerabilities on Docker Hub. In *Proceedings of the Seventh ACM Conference on Data and Application Security and Privacy (CODASPY)*, pages 269–280. ACM, 2017.

[52] SLSA Community. SLSA specification, version 1.2. Supply-chain Levels for Software Artifacts. Status: Approved. Accessed 2026-10-28.

[53] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3687–3706. USENIX Association, 2025.

[54] Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, Massimiliano Di Penta, Daniel M. German, and Denys Poshyvanyk. BOMs away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*, 2024.

[55] The Linux Foundation SPDX Project. The software package data exchange (SPDX) specification. Current version 3.0; the SPDX specification is an international open standard (ISO/IEC 5962:2021). Accessed 2026-10-28.

[56] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410. USENIX Association, 2019.

[57] Marat Valiev, Bogdan Vasilescu, and James Herbsleb. Ecosystem-level determinants of sustained activity in open-source projects: A case study of the PyPI ecosystem. In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 644–655. ACM, 2018.

[58] Boming Xia, Tingting Bi, Zhenchang Xing, Qinghua Lu, and Liming Zhu. An empirical study on software bill of materials: Where we stand and the road ahead. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 2630–2642, 2023.

[59] Nusrat Zahan, Parth Kanakiya, Brian Hambleton, Shohanuzzaman Shohan, and Laurie Williams. OpenSSF scorecard: On the path toward ecosystem-wide automated security metrics. *IEEE Security & Privacy*, 21(6):76–88, 2023.

[60] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. What are weak links in the npm supply chain? In *2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 331–340. IEEE, 2022.

[61] Ahmed Zerouali, Tom Mens, Gregorio Robles, and Jesus M. Gonzalez-Barahona. On the relation between outdated Docker containers, severity vulnerabilities, and bugs. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 491–501. IEEE, 2019.

[62] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. Small world with high risks: A study of security threats in the npm ecosystem. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 995–1010. USENIX Association, 2019.

A Worked Example: etcd Relocation

Serialised event. From the fixture `data/fixtures/etcd-move/` (repository metadata probed on 2026-10-06), the analyst emits a relocation event. Its serialised type is `OWNERSHIP_CHANGE`, a schema name that predates this report. The event asserts only that the recorded path now resolves elsewhere; as Appendix D states, path divergence does not establish a change of controlling principals, and the name should not be read as one. A relocation-specific type is a schema change we have not yet made.

```
affected_artifacts: [{kind: source-repository, ref: github.com/coreos/etcd}]
scope: {kind: project, versions: []} confidence: EMERGING impact: REVIEW
```

In the typed model the scope is the project element `etcd`, and the repository `github.com/coreos/etcd`, carried in `affected_artifacts`, is a *named example*: it illustrates the change rather than bounding it (Section 5). χ is partial: relocation events do not declare completeness, and none of the verdicts below is a negative.

Three verdicts by Algorithm 1.

- `github.com/coreos/etcd`: the named repository gives M, so the verdict is AffectsArtifact (confidence Emerging).
- Package `etcd` with no version: the named repository gives 0 (a named example does not relate to another reference kind) and the project element gives P, so the verdict is AffectsProject.
- `github.com/etcd-io/etcd`: the named repository gives 0, since a named example says nothing about other repositories. The new canonical path does *not* resolve to project `etcd`: it is an unmapped reference that resolves to itself. Its identity is unresolved, so the project element gives ?, and the verdict is Related.

Specification and implementation agree on all three: the oracle of Section 6 returns the same verdicts for this event, and `test_golden_etcd_namespace_move` pins the implementation's. Earlier drafts modelled the repository as an exact scope element, under which the specification would give RELATED for the package and the case appeared as a discrepancy. That reading predates correction S4, which fixed the role of named examples; it is not a remaining difference, and the only one at the audit snapshot or the release is the label difference of Section 6.

Unjustified negative, unresolved result, justified exclusion. Table 23 traces the three outcomes the report distinguishes through one event, Bitnami's catalogue-wide withdrawal (`data/fixtures/bitnami/event.json`), plus one unmapped reference. The canonical identity of every reference is computed before any test (D9). For each verdict OPENPULSE retains an evidence contract. It carries the event and its cited sources, the identity with its trust status, the scope, the matching element and match method, the assessment, the unknowns, the temporal roles, and a content hash. The first detection is also recorded in the ledger.

Reference and scope	Element tests and completeness	Verdict	Outcome
`bitnami/nginx:1.25` against the five-image list read as complete	Every listed image gives N; χ assumed, no basis	NOTAFFECTED (at `a461bc9`)	Unjustified negative (D4)
`docker.io/zzz/kvstore:3.5` against an etcd 3.5 end-of-life scope	Identity unresolved: the reference resolves only to itself, so it gives ?	RELATED	Unresolved, correctly
`docker.io/redis:7.2` against the namespace elements under `docker.io/bitnami`, `docker.io/bitnamilegacy` and the chart registry	Every namespace element gives N by location; χ declared, announced basis	NOTAFFECTED	Justified exclusion, under a policy admitting announced scopes

Table 23: Three outcomes across two events. At `8a9c3a8` the same Bitnami scope makes `bitnami/nginx:1.25` AFFECTSARTIFACT, through the exact element `docker.io/bitnami/*:<version>`, and `bitnami/redis:latest` RELATED, through the envelope `docker.io/bitnami/*`.

B Defect Details

D4 in curated data. Implicit completeness also affected the curated evidence. The Bitnami event's scope listed five *example* images of a catalogue-wide change, and because the list was read as complete, an unlisted image such as `docker.io/bitnami/nginx` was cleared. Marking the list partial would have made every container image RELATED to that event. The correct repair was to state the scope the announcement actually describes, using namespace elements declared complete. At `8a9c3a8` they are `docker.io/bitnami/*:<version>` (exact) and `docker.io/bitnami/*` (envelope), `docker.io/bitnamilegacy/*:*`, and the chart registry `bitnamicharts/*:*`. With that scope, `bitnami/nginx` is affected, `bitnami/redis:latest` is RELATED, and `docker.io/redis` is cleared.

D7: tags. Artefact elements match on the repository triple and ignore tags. For the Bitnami event, which names `bitnami/redis:7.2` among its affected artefacts, the engine therefore reports `bitnami/redis:5.0.4` as "matching" `7.2`. That outcome is correct only because the event is repository-wide. It also reports `bitnami/redis:latest` as affected, although `latest` is what remained in the mainline. The result was a false impact with an overstated reason, not a false clear.

The repair in bfb1ee9 has three parts. Element roles make the plain namespace an envelope. The tag predicate <version> restates the exact part of the scope. And a named literal tag now establishes that tag only, so bitnami/redis:latest is RELATED and bitnami/redis:5.0.4 is affected through the versioned-tag element.

D8: numeric pre-release identifiers. The generic scheme splits a version into digit and letter runs. A numeric component after - or + is therefore folded into the numeric core: 1.0.0-1 becomes $(1, 0, 0, 1)$ and sorts *after* 1.0.0. Under SemVer, 1.0.0-1 is a pre-release and sorts *before* 1.0.0. Against an OSV SEMVER range with introduced: 0, fixed: 1.0.0, the versions 1.0.0-0 and 1.0.0-1 are reported outside the range although they are inside it. That is a false clear. The same folding makes 1.0.0+5 (build metadata, which SemVer ignores for ordering) sort above 1.0.0, and it treats Debian revisions such as 2.4.1-1 as an extra numeric component. The scheme also ignored the number after a marker, so 1.0.0-rc1 equalled rc2, and a fix in rc2 cleared rc1. The repair in 9d53ce2 is a closed grammar: a version is comparable only as a dotted numeric core followed by at most one recognised marker and an optional number. Anything else, including 1.0.0-1, 2.4.1-1 and 1.0.0+5, is undetermined, and rc1 sorts below rc2. That conservative repair was an intermediate state for SemVer ranges: since D11 (below), an OSV SEMVER range is compared by SemVer precedence, under which 1.0.0-1 is inside the range, as it should be. The closed grammar still governs the generic scheme. D8 was not caught by the range tests, which we wrote from our own reading of the specification. It is the kind of error that the version differential (Section 6) catches: its mutant is caught there.

D9: registry host aliases. Reference canonicalisation lowercases a reference, strips a URL scheme, a digest and the tag of the last path segment, defaults a missing registry to docker.io, and gives a single-segment name the namespace library. It did not fold the aliases under which Docker Hub also serves images, index.docker.io and registry-1.docker.io, into docker.io. Against the Bitnami event's namespace elements, declared complete, docker.io/bitnami/nginx was affected while index.docker.io/bitnami/nginx, the same image, was cleared. That was a false clear. Exact comparison is only as meaningful as the canonicalisation in front of it. The specified repair is a canonical form per element kind: registry host aliases, namespace and library, package-name normalisation per ecosystem, curated aliases, and digest algorithm. The executable oracle and the conformance suite need the same form. The fix in 7c0b9d9 folds both Docker Hub hosts into docker.io in the one normaliser that matching and identity share. Package-name normalisation per ecosystem is not part of the canonical form at 8a9c3a8; it was added after the audit (F1–F3, Appendix C).

D10: unmapped identity. An unmapped reference resolves only to itself, and the identity layer marked that self-resolution VERIFIED, as if it established a different project. Two paths followed. Under a version-scoped event, such as an end-of-life notice for etcd 3.5, an unmapped mirror docker.io/zzz/kvstore:3.5 was excluded on identity grounds and cleared per pair. Under a project-wide event it was UNKNOWN, unless its name resembled the project, and dropped from the summary, so another event's clearance headlined. The fix in 7c0b9d9 makes every self-resolved reference ?, whatever its name. Three regression tests had used unmapped references to stand for "another project". They now use references curated as other projects. One test asserted the name rule itself, and it is reversed.

D11: pre-release ordering under SemVer. The generic grammar ranks pre-release markers by a fixed table: dev below a/alpha, below b/beta, below c/rc/pre/preview. It treats a and alpha, and c and rc, as equal, as PEP 440 does. SemVer orders alphanumeric pre-release identifiers lexically instead. So 1.0.0-dev sorts *above* 1.0.0-beta, a below alpha, and c below rc. The generic comparator is applied to OSV SEMVER ranges as well, and three false

clears follow. For example, `1.0.0-dev` against `introduced: 1.0.0-beta, fixed: 1.0.1` is reported outside the range although SemVer places it inside. Accepting a form is therefore not enough. The form must be supported under the range's own comparison scheme, and its ordering must agree with that scheme. The repair in `dabafef` dispatches on the range type and ecosystem, and returns `?` wherever no implemented scheme reproduces the ecosystem's ordering (Section 5).

D12–D14: found by the conformance matrix. D12: a reference whose first segment is a registry host was treated as a registry reference only with three or more segments. So `docker.io/redis`, Docker's own spelling of `docker.io/library/redis`, was read as namespace `docker.io`, and complete scopes naming the official image cleared it. D13: an unmapped package was dropped as unrelated to other projects' events, the package-side twin of D10. It is now unresolved against project-wide, release-line and package scopes, and has no association with container-image and registry scopes. Two packages that tests had used to mean "a dependency nothing applies to" are now curated as their own projects. D14: a repository element that names no tag gave RELATED for a digest-pinned reference to the repository, though it covers every pin. All three are fixed in `784fed8`.

D16: untrusted mappings drop associations (open). Found at the release while checking Table 29 against the code. A heuristic or REVIEW-REQUIRED mapping to another project yields no association (UNKNOWN, left out of the summary) where Section 5.3 requires ?. On the image path this happens for project-wide and package-scope events: the identity check returns RELATED only for a self-resolution. On the package path the curated-other guard ignores the trust status, so it also covers release lines. It is reachable with the shipped catalogue: `docker.io/bitnami/postgresql:16` resolves by the namespace heuristic and is UNKNOWN against a project-wide PostgreSQL event, and nine catalogue entries are REVIEW-REQUIRED. No pair verdict becomes NOTAFFECTED, but a summary can show "cleared" over an association it should keep, the failure of D10's first reproduction. D5's fix stopped heuristics from excluding but not from disassociating. The conformance matrix cannot see it, because its oracle has no untrusted identity. The fix (return ? unless the mapping is curated and VERIFIED) is not in the release.

A latent gap: packages against source repositories (open). A Go module curated as another project but located in a source repository the event names gets no association, in the code and in the oracle, though Table 29 requires ?; P2 fails under a correct mapping. The code also gives no association for any package against a source-repository *bounding* element, where the oracle gives ?. Neither is reachable at the release: no Go module is curated, and no producer puts a source repository in a bounding scope. The matrix has no Go dependency and no source-repository element, so neither case, nor D-R1, is inside the conformance claim.

C Artifact Manifest

Every statement about behaviour, and every measurement, refers to one release. The frozen data sets and the audit history below are states of the method, not further implementation versions.

Release.

- Release v0.8.0 (git tag `v0.8.0`, 31 October 2026) of `https://github.com/IkarusAILabs/OpenPulse`; licence Apache-2.0. The measurements were taken at commit `787beb82845ac56ff094b4d53b856b3fdd567874`. Outside

documentation and this report, the release differs from that commit only in its version number (`pyproject.toml`, `core/version.py`).
- Evidence contract 1.2.0; intelligence schema 0.7.0; Python ≥3.10 with the hash-locked `requirements.lock`. All tests run offline: 1,162 collected at the release, as run by CI. Test count is not evidence of semantic completeness.
- Conformance matrix: `python conformance/matrix.py`. Expected: 6,195 cases, 6,123 conform and 72 differ by one documented label. Results, including every disagreement, are in `conformance/results/` and `conformance/REPORT.md`. The build gate is `tests/test_conformance_gate.py`.
- Randomised: `python conformance/randomized.py -cases 100000 -seed 7`. Expected: 100,000 summaries and 200,070 per-event verdicts conform.
- Re-injection: `python conformance/mutation.py`. Expected: 22 of 22 mutants caught, also without the regression tests.
- Version differential: `python conformance/versions_differential.py`. Expected: PEP 440, 233,289 pairs, 224,676 agree, 6,685 abstain, 1,928 reference-invalid, 0 disagree; PyPI ranges, 7,110 agree; SemVer vectors pass. The npm, Go and Maven bridges need the pinned toolchains and run in CI.
- Cross-event aggregation: `python papers/tr-2026-01/check_aggregation.py` checks the summary function against Table 9. Expected: 19,607 sequences, 13,699 set orderings, 0 mismatches.
- Cost: `python benchmarks/cost.py` (offline), with `-live` for acquisition; results and machine in `benchmarks/REPORT.md`. The history-length row of Table 16 is not yet a benchmark option.
- Evaluation harness (Appendix F): `python evaluation/run.py` reproduces the development-set illustration. The registered run adds `-require-comparators -require-encodings`, with `-inventories` and `-annotations` for W1–W3 and D1–D2 (`evaluation/REGISTRATION.md` §9). Of the factorial cells, only T_K and X_0 are implemented; L_0, L_K, X_K and T_0 are not, so the registered run cannot yet be made.
- Contract hash: `verify_v1_content_hash` in `core/evidence_contract.py` recomputes it with the hash and timestamp fields pinned to constants (`docs/EVIDENCE_CONTRACT.md`).
- Case ledger: `pytest tests/test_golden.py`; the recorded evidence is under `data/fixtures/`.
- Open: D16 and the latent package–source-repository gap (Appendix B) are not fixed and have no regression test.

Frozen data sets.
- Retrospective study (Section 10): implementation frozen at tag `retro-freeze-2026-10-30` (`3639a0f`); case file `evaluation/retrospective/cases/retro-2026-11.json`, SHA-256 `70832760…a1bf83`, checked in CI before any run; results `results/openpulse-retro-2026-11.json`, SHA-256 `2bce9237…ed1376`, reproduced byte for byte by a second CI run.
- After-fixes run (diagnostic): the same case file on `abcb685`, the merge of the freeze tag with the D-R1 and D-R2 fix `7744ca0` and no other code change; results in `results/after-fixes/`, SHA-256 `ea8d772b…88014bb2c`.

Audit history.
- Audited commit: `a461bc99a6edebbdb124e4b9a3c477282aae45a6`. Audit snapshot: `8a9c3a829ad0da7a8a1e935f677af0eb8810ed0c` (`main`, 29 October 2026), evidence contract v1.1; its full suite of 953 tests passes. D1–D3 and D5 were fixed in `4b92f7d`, D6 in `b5cbbde`, D4 and the interim rules in `f0c3b48`, D8 in `9d53ce2`, D9 and D10 in `7c0b9d9`, D11 and the admission conditions in `dabafef`, D7 and the element roles in `bfb1ee9`, D12–D14 in

784fed8, and D15 in d4f7d1f.

- Audit harness: the regression tests do not exist at the audited commit, so they are transplanted. The script papers/tr-2026-01/reproduce_audit.sh does this from a clean clone. It checks out a461bc9 and 8a9c3a8 as separate worktrees, copies tests/test_false_clear.py from 8a9c3a8 into the audited one, and runs it in both. Expected result: at a461bc9, 52 tests fail (the defect, interim-rule, admission and role tests) and 12 pass; the passing ones pin behaviour the fixes must preserve. The withdrawn-advisory test passes at a461bc9 only because D2 hides it. At 8a9c3a8 all 64 pass. Table 24 gives the breakdown by defect.
- D8: osv_applicable("1.0.0-1", {"ranges": [{"type": "SEMVER", "events": [{"introduced": "0"}, {"fixed": "1.0.0"}]}]}) in core/versions.py returns False (cleared) at f0c3b48, None (undetermined) at the D8 repair 9d53ce2, and True (inside) from the D11 repair dabafef on, including 8a9c3a8 and the release.
- D9: check_dependency({"kind": "image", "ref": "index.docker.io/bitnami/nginx:1.25"}, [event]), with the event from data/fixtures/bitnami/event.json, returns NotAffected at 9d53ce2 and AffectsArtifact at 8a9c3a8, like its docker.io spelling.

Defect	Tests in test_false_clear.py	Fail at a461bc9	Fixed	Residual limitation
D1	test_d1_*	4 / 4	4b92f7d	OCI levels unresolved (interim ?)
D2	test_d2_*	4 / 9	4b92f7d	GIT and unknown range types undetermined
D3	test_d3_*, test_package_scoped_*	2 / 4	4b92f7d	–
D4	test_d4_*	3 / 4	f0c3b48	Declarations unverified
D5	test_d5_*	2 / 2	4b92f7d	A wrong curated mapping is not detected
D6	test_d6_*	3 / 3	b5cbbde	Epochs undetermined, not compared
Interim	test_interim_*, test_namespace_*	7 / 9	f0c3b48	OCI resolution open
D7, roles	test_d7_*, test_envelope_*, test_version_tag_*, test_bitnami_*	5 / 5	bfb1ee9	Roles by form, not labelled in the contract
D8	test_d8_*	4 / 4	9d53ce2	One generic scheme; ecosystem forms undetermined
D9	test_d9_*	2 / 2	7c0b9d9	Package names folded for PyPI and Maven only
D10	test_d10_*	4 / 4	7c0b9d9	Coverage cost unmeasured; curation needed
D11	test_d11_*	3 / 3	dabafef	Other ecosystems' orderings undetermined
N3–N5	test_n3_*, test_n4_*, test_n5_*, test_package_name_*	6 / 7	dabafef	Freshness opt-in in the library call only; one bound on the newest item; cross-event conflicts
D12–D14	test_d12_*, test_d13_*, test_d14_*	3 / 4	784fed8	Uncatalogued packages now Related
Total	64 tests	52 / 64		

Table 24: Audit reproducibility by defect. Passing tests at a461bc9 pin behaviour that the fixes must preserve. Residual limitations are those at release v0.8.0.

Changes after the audit snapshot. Table 25 lists the changes made after the audit snapshot 8a9c3a8. All are included in release v0.8.0.

Change	Commit	What it does	Error it removes
F1: identity per ecosystem	by `a1ecc58`	A package with a stated ecosystem resolves to a project only through a curated (ecosystem, name) pair (one with none, through a curated bare-name alias); every catalogue entry now lists its packages. PyPI `redis` (redis-py) had been treated as the Redis server	False impact
F2: project package element	by `a1ecc58`	A package-scope value that names the event's own project denotes every package curated as that project; previously applied but unspecified	Specification gap
F3: qualified package values	by `a1ecc58`	A purl value (`pkg:pypi/redis`) decides by ecosystem and name before identity	Name-only matching
F4: self-resolved packages	by `a1ecc58`	A package that resolves only to itself is RELATED even when named like the project (the package-side twin of D10)	Name taken as identity
Contract 1.2.0	by `a1ecc58`	Adds `verdict_meaning` (direct scope; embedded components, availability, maintenance and security consequence marked not evaluated), `scope.completeness` with its basis, and the evaluation context	Bare labels read as permission
D-R1, D-R2	`7744ca0`	A Go module reaches the source repository it lives in (positively only); the GitHub collector follows one same-origin redirect, so transfers are observed (Section 10)	Silent miss; missed relocation
D-S1–D-S3	`98500d4`	OSV advisories without a CVE are kept; another package's range in the same advisory is ignored; version-only entries follow the OSV schema (Section 10)	**False clear**; false impact
Scoped npm identity	by `a1ecc58`	`@scope/name` keeps its scope instead of resolving to the empty string	Shared empty identity
Discovery	by `a1ecc58`	Targets come from the inventory (Go modules, repository references, Docker Hub images), and every npm, PyPI, Maven, Go and crates.io package is queried on OSV by name	Uncatalogued projects unseen
Catalogue-map cache	`0174a26`	The package, curated-slug and alias maps are memoised per catalogue object instead of rebuilt on every identity resolution. Since F1 that rebuild took about three quarters of matching time: matching rose from about 42 µs per pair at `8a9c3a8` to about 75 µs at `a1ecc58`, and is about 25 µs with the cache. Every decision is unchanged	1.8× matching slowdown from F1
Agent freshness bound	`323bb3c`	The agent tools run in a *current* mode with a 90-day evidence-age bound by default, or an explicit *replay* mode without one that says so; responses carry the evaluation context and each cause's scope completeness	Agent negatives on evidence of any age
Evaluation	by `a1ecc58`	Strong baseline (v) with pinned comparators, an independent oracle kit, a registration draft and the retrospective study	–
Registered analysis	`56b88e5`	The existing-representation baseline (vi) with written encoding rules; tests S3 (then defined against (vi), X_0) and G4; the burden (W1–W3) and input-reliability (D1–D2) measures; the stopping rule and inclusion criteria (Section 9)	–
Freshness everywhere	`787beb8`	The command line's verdict commands run in the agent tools' current mode by default (90-day bound; `-replay` drops it and says so); adversarial-input tests (Table 19); evaluation strata by completeness basis and identity status, pair-level false-clear bounds, deduplicated review items	CLI negatives on evidence of any age

Table 25: Changes made after the audit snapshot `8a9c3a8`, all included in release v0.8.0. *Commit* is where each change landed. Each has regression tests; the oracle, the conformance matrix, the randomised generator, defect re-injection and the strong baseline were extended for F2–F4.

Appendices. Appendix A is a worked example; B gives defect details; C is this manifest; D is the development case ledger; E holds designs not yet implemented; F condenses the registered evaluation; and G is the normative membership table.

#	Class	Observed change	First det.	Effective (provenance)
2	Distribution	`bitnami/redis`: 534 tags, 0 version-like; `bitnamilegacy/redis`: ≥919 version-like	2026-10-02	2025-08-28 (official notice)
6	Relocation	`fireeye/capa` → `mandiant/capa`	2026-10-06	unknown
7	Relocation	`coreos/etcd` → `etcd-io/etcd`	2026-10-06	unknown
8	Relocation	`containous/ traefik` → `traefik/traefik`	2026-10-07	unknown
9	Relocation, archive	`facebook/ react-native-fbsdk` → `facebookarchive/`…; archived	2026-10-07	unknown (last push 2021-03-26)
10	Distribution	`bitnami/spark`: no tags listed; `bitnamilegacy/spark`: 936 tags	2026-10-07	2025-08-28, postponed to 2025-09-29 (official notices)

Table 26: Development ledger (numbers follow `docs/DISCOVERIES.md`). Detections came from single-day probes. Pipeline confidence is EMERGING throughout.

Reference	Verdict	Observed listing	Consequence	Evidence missing
`bitnami/spark:3.5`	AFFECTS-ARTIFACT	no tags listed	Withdrawal; migrate	Manifest resolution
`bitnamilegacy/spark:3.5`	AFFECTS-ARTIFACT	tag listed	Frozen distribution; no updates	Manifest resolution; whether fixes land
`apache/spark:3.5`	NOT-AFFECTED	not observed	None from this event	– (declared-complete scope)

Table 27: Case 10: membership is not consequence.

D Development Case Ledger

Before collecting cases we set an acceptance target: at least five non-lifecycle changes, detected by the pipeline and replayable offline, with any unreproducible candidate retracted. Each case is pinned by a golden test that replays the recorded raw evidence through the production analysts. The author found the cases while developing the detector, so the ledger shows *detectability*, not effectiveness.

Relocations (Cases 6–9). The detector compares the queried path with the canonical path GitHub returns, and fires when the owner differs. It names the old path as the affected artefact. It observes path divergence. It does not observe a change of controlling principals, which can also happen with no path change. Nor does it show that the old path can be reclaimed [18]. Same-owner renames are silent by policy, which leaves a coverage gap.

Distribution (Cases 2 and 10). Case 2 is treated as a change only because official announcements say versioned images moved. The observation corroborates the announcement. Two different completeness requirements apply. The claim that the mainline has *no* version-like tag needs a complete mainline listing, and the ledger records a full-set observation of its 534 tags. The claim that the legacy namespace *has* versioned tags needs only one example, so the truncated lower bound of ≥919 suffices. Removal inference is withheld on truncated sets because removal is a claim about absence. Case 10 rests on tag listings from two endpoints. No reference-specific manifest resolution was performed, so Table 27 reports listings and makes no claim about whether a reference resolves.

Retracted candidates. On 2026-10-01 the pipeline reported tags disappearing from four official images. Three were formalised as Cases 3–5. One mechanism explained all four: the

Case	Announced	Effective	First det.	Lead (d)	Catalogued
Retrospective: observed historical changes					
2 Bitnami split	2025-07-16	2025-08-28	2026-10-02	−400	no
10 Bitnami Spark	2025-07-16	08-28 or 09-29	2026-10-07	−405 / −373	no
6–9 Relocations	unknown	unknown	2026-10-06/07	undefined	no
Prospective: scheduled lifecycle events					
PostgreSQL 14 EOL	policy page	2026-11-12	2026-10-06	37*	yes
Redis 8.0 EOL	policy page	2026-12-01	2026-10-06	56*	yes

Table 28: Lead times. *Catalogued*: whether an existing lifecycle or vulnerability catalogue already records the change. All first detections came from single-day probes, so there was no monitoring start, polling policy or acquisition-failure record. Effective dates are as cited and were not independently verified. *Relative to the current schedule.

probe read only the five most recent tags, so a tag pushed out of that window looked removed. Every candidate was present in the full listings. The fix added full pagination, a `truncated` flag that withholds diffs, and a parser-version re-baseline.

Lead times. Table 28 applies the definition of Section 4.4. All non-lifecycle detections were late, and the two prospective cases belong to a class that catalogues already record. No advance warning for an unrecorded change class has been observed.

Golden scenarios. Eleven scenarios (Bitnami, iText, MinIO, Django, Redis, PostgreSQL, Kubernetes, Kafka, cert-manager, Grafana, Terraform) fix expected identity and applicability outcomes. They are regression tests, not measurements, and four of them had encoded D5.

E Designs Not Yet Implemented

This appendix describes two designs that are not implemented at the release v0.8.0. Nothing in it is evidence.

Cost of resolving OCI levels and tags. Multi-architecture images make the interim OCI rule of Table 15 matter in practice, so its full resolution is the next implementation step. It need not query manifest lists for every dependency. Resolution is needed only when a digest-pinned dependency meets a digest-bearing element of the *same* repository, which is the only case the interim rule leaves undetermined. Tag-pinned dependencies and digests of other repositories never need it. Each resolution is one manifest fetch by digest. What may be cached indefinitely is immutable content: fetched manifest bytes whose digest has been verified, and the index-to-manifest relation they state. Mutable accessibility may not be cached indefinitely. Whether the registry still serves the object, whether the consumer is authorised to pull it, and what a tag points to all need a time limit. With that split, the content cost is one request per distinct digest ever seen, not one per dependency per run. Recursion into nested indexes is bounded. Tag mutability is a separate cost. Matching tags against scope elements needs no registry call. Resolving a tag to the digest it currently names is needed only to compare it with a digest element, and that answer expires. It is therefore cached with a time limit and an observation timestamp, and a failed lookup leaves the verdict undetermined. Caching bounds the cost of repeated references. It does not bound the cost of previously unseen digests, of nested indexes, or of registry unavailability, so it is not by itself an argument that the design scales. We have not measured these costs.

Injection mitigation for the agent interface. The aim is structural separation, so that an agent can make its decision without ever reading upstream text as part of an instruction-bearing field. We specify four measures, none of them implemented yet. (i) *Decision fields are closed vocabularies.* Relationship, affected flag, confidence, identity status, event class, completeness state and dates are enumerations or validated formats. The tool description tells the agent to decide from these fields alone. (ii) *Upstream strings are quarantined.* Every upstream-derived string moves into a single block labelled `untrusted_upstream`, together with its source and provenance. Strings are length-bounded and stripped of control and markup characters, and identifiers are checked against the charset rules the collectors already enforce. Nothing outside that block contains upstream text. (iii) *Reasons are templated.* Verdict reasons are generated from fixed templates whose slots hold only validated identifiers. They do not quote upstream content. (iv) *Testing.* The agent experiment designed in `evaluation/FURTHER_STUDIES.md` includes an injection condition, and a corpus of hostile repository names, tags, URLs and licence strings is run against the server in regression tests. Sanitising text cannot make an LLM immune to injection. Labelling is not isolation either. A quarantined block in the same model-visible response still reaches the model's context, and nothing makes the model ignore it. What the design achieves is narrower: the evidence an agent needs never *requires* reading untrusted text. The decisive question is whether hostile content reaches the decision context and changes behaviour, and that is empirical. The agent experiment therefore compares three response forms. The first is structured-only. The second is structured, with raw upstream evidence retrievable only through a separate tool call. The third is mixed, with the untrusted content labelled. Only the second keeps untrusted text out of the context by default.

F Registered Evaluation

This appendix condenses the registration draft `evaluation/REGISTRATION.md`, which remains the canonical text; § numbers below refer to it. Nothing here has been run on held-out data. The draft becomes a registered protocol only when a team archives it (OSF Registries or Zenodo) with its registry identifier, freeze commit, event window and annotators filled in, before any held-out label exists. That has not happened. Before the evaluation, the evaluated subset must enforce the full negative-admission policy, or abstain wherever it cannot; the evaluated configuration sets the freshness bound.

Scope of the claim. The evaluated OpenPulse receives events and no advisory data, so the study concerns event-scope decisions only. The advisory path, which correlates OSV and NVD advisories by package and version, never runs. It is where the retrospective study found a false clear (Section 10). It needs its own registration and benchmark, with advisories frozen as they stood on each event's date, including advisories without a CVE identifier, multi-package advisories, withdrawn advisories and version-only entries (`evaluation/advisory/DESIGN.md`), reporting false clears, silent misses, false impacts and unresolved decisions. That benchmark is designed, not run: no snapshot, case or label exists, so the advisory path has regression tests and no measurement. No result of one domain is claimed for the other.

Unit and annotation (§7–§8). The unit is a (dependency, event) pair. Pairs that share an event or a project are correlated, so intervals are clustered by event, a sensitivity analysis clusters by project, and every result reports the number of independent events and projects next to the pair count. Every method receives the same event evidence and inventory. Two annotators, blind to every method's output, label each pair as in scope, out of scope or undeterminable from the event's cited sources and the registry or repository state frozen at a recorded date. The announcing source defines the intended scope; registry and repository observations establish current state; an unresolved conflict is undeterminable, never forced into a binary label. A

third annotator adjudicates, and agreement is reported as Cohen's κ. Every pair is labelled, including those OpenPulse never associates. Undeterminable pairs are never false clears or false impacts; they feed the *unsupported-decisiveness* rate, the share given an unqualified Affects or NotAffected.

Metrics. With F the truly affected pairs labelled NotAffected, A the truly affected pairs and C the NotAffected predictions, $\mathrm{FCR}_{\mathrm{affected}} = F/A$ and $\mathrm{FCR}_{\mathrm{clear}} = F/C$. The first has the same denominator for every method; the second is undefined when $C = 0$ and is then reported as such. Safety is stated over independent events: with n_e events and none containing a false clear, the one-sided 95% upper bound on the per-event failure probability is about $3/n_e$. Let P be the evaluated pairs, $o(p)$ the outcome for pair p, and $D = D_{\mathrm{in}} \cup D_{\mathrm{out}} \subseteq P$ the determinable pairs, labelled in or out of scope. Write $\mathcal{M}$ for membership in an exact element (AffectsArtifact, AffectsVersion or AffectsPackage); every NotAffected is admissible. *Decisive coverage* measures willingness to decide, regardless of correctness:

$$\text{coverage} = \frac{|\{p \in P : o(p) \in \mathcal{M} \cup \{\text{NotAffected}\}\}|}{|P|}. \tag{2}$$

Resolution combines decisiveness with correctness, over determinable pairs only:

$$\text{resolution} = \frac{|\{p \in D_{\mathrm{in}} : o(p) \in \mathcal{M}\}| + |\{p \in D_{\mathrm{out}} : o(p) = \text{NotAffected}\}|}{|D|}. \tag{3}$$

Resolution is the primary usefulness endpoint (H2); decisive coverage is reported and never tested. A policy that answers everything has coverage 1 and a resolution reduced by every wrong answer. Unknown, AffectsProject, Related and abstention for stale or unavailable evidence count in neither numerator. Precision and recall of *affected*, the false-impact rate, and the Unknown and Related rates are reported by event class, match type, evidence strength, identity status, completeness state and basis, and confidence level, with the share of unresolved pairs that adjudication finds truly affected. These metrics are read together: a low false-clear rate is not enough if most hard cases become unresolved, and high coverage is worthless if it comes from unsound exclusions.

Decision criteria (§3–§5). There is one primary endpoint, at one operating point: the implemented policy $\mathcal{A}$, with no minimum confidence. Two primary hypotheses must both hold. *Safety (H1)* is an absolute bound, not a comparison: the exact one-sided 95% Clopper–Pearson upper bound on the share of events in which OpenPulse issues a false clear is at most $\delta = 0.05$. H1 is decidable only with at least 60 events at risk; with fewer the result is "not decidable", never a pass. At 60 events, one false-clear event already fails H1 (bound 0.077). Earlier drafts subtracted the share of the minimal reference (iv), but (iv) never clears, so that share is zero by construction and the comparison was nominal. The false-clear events and bounds of every baseline are reported beside H1. *Minimum usefulness (H2):* the paired difference in resolution (not decisive coverage), OpenPulse minus (iv), is at least 0.10, with an event-clustered bootstrap one-sided 95% lower bound above zero (4,000 resamples, fixed seed). H2 shows only that the model's clearing rule resolves more than a policy that never clears; it is not a comparison with careful practice. The practical claim rests on the existing-representation comparison with the same identity knowledge, OpenPulse against X_K (Table 21), tested as S3, and the report states the S3 result next to the primary verdict, in the abstract and in the conclusion, whatever H2 shows. Secondary tests are Holm-corrected and never rescue a failed primary result: equivalence of OpenPulse and (v) within ± 0.05 by two one-sided tests (S1); the false-clear excess of the naive baseline (S2); and S3. A gain over X_K that comes with more false-clear events than X_K is reported as a trade-off, not as an improvement. The other contrasts of Table 21, including OpenPulse against X_0, are reported with intervals and are not tested. Four guardrails are

reported beside the verdict without changing it: silent misses at most 0.02 of in-scope pairs (G1); OpenPulse's unresolved rate (Related or AffectsProject, over determinable pairs) at most (iv)'s plus 0.05 (G2); every class with an unresolved rate above 0.60 flagged (G3); and OpenPulse's median unresolved references per inventory and weekly cycle no higher than (iv)'s (G4). The registration states which of these values came from judgement. A result that improves resolution or coverage only by raising the false-clear rate beyond the margin counts as a failure.

Baselines (§2). The baselines are: (i) direct metadata rules; (ii) namespace-aware matching without completeness or identity caps; (iii) curated feeds with SCA matching; (iv) a *minimal conservative reference*; (v) a *strong conservative baseline*; and (vi) an *existing-representation baseline*. Beating (ii) is expected by construction. Algorithm 2 gives the decision rule of each baseline that is run, as implemented or specified in `evaluation/`; it is the only definition, and the text below only explains it. The minimal reference (iv) is policy B of the harness. It reports Affects when the dependency is literally listed by a current event and abstains otherwise, so it *never clears*. Its identity is by bare name, the way an affected-product list is commonly read: a package's or repository's own name against the event's project and listed packages, after Docker Hub aliases are folded. Versions match by release-line prefix. (Earlier drafts described (iv) as admitting negatives from scopes in $\mathcal{A}$ with exact identity; no such policy was implemented, and the retrospective study ran the policy described here. That earlier policy is now cell T_0 of Table 21.) (iv) is the comparator for H2 and the floor for G2 and G4, not a test of the model.

The strong baseline (v) uses the same scope language, admissibility policy, curated mappings, event-confidence and freshness policies, and unknown and contradiction information as OpenPulse, with the ecosystems' own version comparators and conservative handling of every unsupported relation. The completeness condition, the identity rule and the admission policy are therefore *shared*, and (v) measures only what OpenPulse's matching code adds. That residue can be stated in advance. (v) relates a reference to an element only through the same reference kind, a curated mapping, or the project element; OpenPulse additionally relates a Go module path to the source repository it names (D-R1), and reads the release from an image tag with a variant suffix (`node:12-alpine` as release 12), where (v) compares image tags only as plain dotted numbers. (v) is therefore the comparison that tests whether the engine adds anything (S1).

For (vi), each held-out event is re-encoded, from its primary sources alone and under frozen written rules, as an OSV record or as OpenVEX statements. The encoder sees no labels or policy output. A conservative consumer then reads the encoding with the same pinned comparators as (v). It reports NotAffected only from an excluding OSV range or an explicit `not_affected` statement, and abstains otherwise. It has no typed namespace elements and no completeness declaration. As implemented, it also has no catalogue: identity is one exact purl (X_0). Because (v) and OpenPulse do receive the catalogue, the draft also runs (vi) with the same frozen catalogue (X_K): a statement or OSV package matches if it names any identifier that a curated, verified mapping ties to the dependency. X_K differs from (v) only in representation. This is representation (2) of Table 2, read with care. The encoding rules (`evaluation/representations/ENCODING.md`) fix the format per kind of change, one purl construction shared by encoder and consumer, release-line boundaries per ecosystem, and seven loss codes. What a format cannot express, such as "every image under this namespace", is recorded per event as encoding loss, and (vi)'s results are also reported on events with no loss, so that a poor encoding cannot flatter OpenPulse. Each encoding is frozen by SHA-256 before any policy runs. On the development set, encoded after the other results were known and so only illustrative, (vi) and OpenPulse each resolved 5 of 11 decidable pairs on the three lossless events.

Algorithm 2 Decision rules of the baselines, for one dependency d and the current events C (withdrawn and superseded records removed). $\mathrm{id}_k(d)$ is d's identity under identity knowledge k (Table 21); $\mathrm{id}_{\mathrm{name}}$ is bare-name identity. UNRES is an abstention, NONE is no association. SUMMARY returns the first of AFF, UNRES, CLR present, else NONE. L_k is B with id_k in place of $\mathrm{id}_{\mathrm{name}}$. Code: `evaluation/policies.py`, `strong_baseline.py`, `existing_representation.py`; L_0, L_K, X_K and T_0 are not yet implemented.

function LISTED(d, e, id) ▷ literal reading of e's scope values and named artefacts
  return some value v of e names $\mathrm{id}(d)$
  (images: canonical reference, `ns/*` and tag predicates; versions: release-line prefix)
end function
function A(d, C) ▷ naive
  return AFF **if** $\exists e \in C$: LISTED($d, e, \mathrm{id}_{\mathrm{name}}$) **else** CLR
end function
function B(d, C) ▷ (iv): never clears
  if $\exists e \in C$: LISTED($d, e, \mathrm{id}_{\mathrm{name}}$) **then**
    return AFF
  end if
  return UNRES **if** $C \neq \emptyset$ **else** NONE
end function
function X(d, C, k) ▷ (vi) = X_0; X_K with catalogue
  for all $e \in C$ **do**
    $enc \leftarrow$ frozen OSV/OpenVEX encoding of e
    if enc is missing **then**
      $o_e \leftarrow$ UNRES; **continue**
    end if
    $o_e \leftarrow$ AFF if an OSV range for a package in $\mathrm{id}_k(d)$ includes d's version,
    or an `affected` statement names a purl in $\mathrm{id}_k(d)$;
    else CLR if an excluding range or a `not_affected` statement matches;
    else UNRES
  end for
  return SUMMARY($\{o_e\}$)
end function
function T(d, C, k) ▷ (v) = T_K; T_0 with empty catalogue
  for all $e \in C$ **do**
    $R \leftarrow$ element relations of d to e's typed scope under id_k (oracle parser, pinned comparators)
    if R has a match **then**
      $o_e \leftarrow$ AFF
    else if R has an undetermined relation **then**
      $o_e \leftarrow$ UNRES
    else if R excludes d and the negative is admissible **then**
      $o_e \leftarrow$ CLR ▷ χ complete with a basis, current, uncontested, fresh
    else if R is nonempty **then**
      $o_e \leftarrow$ UNRES
    else
      $o_e \leftarrow$ NONE
    end if
  end for
  return SUMMARY($\{o_e\}$)
end function

Comparators and ablations. The ecosystem comparators are pinned: `packaging` (hash-locked) for PEP 440, `node-semver` 7.8.5 for npm, `golang.org/x/mod` v0.42.0 for Go modules and Maven's `ComparableVersion` from `maven-artifact` 3.9.16 (jar SHA-256 checked). Each runs through a bridge that reports its library version, and the analysis refuses to run unless every pin matches. `dpkg` ordering is out of scope. The comparison therefore evaluates matching and decision policy, not the whole intelligence pipeline. The gate, the identity cap, completeness and the coverage qualifier are ablated.

Review burden (§5). Per-pair abstention hides workload, so the draft adds three burden measures for OpenPulse, (v), (vi) and (iv): W1, unresolved references per inventory and weekly cycle, over every reference of each participating inventory, with the actionable share and the unique review items after deduplication; W2, unresolved references per event at risk; and W3, timed reviewer minutes per unresolved item, on a seeded random sample of 50 of OpenPulse's and 50 of (iv)'s unresolved items.

Reliability of trusted inputs (§5a). D1 and D2 are necessary evidence, not supplements. Negatives are sound only if completeness declarations and curated mappings are correct (Table 19), and neither oracle agreement nor H1 tests that: an under-inclusive declaration is a false-clear risk whether or not a sampled pair hits it. D1 has two annotators judge every completeness declaration that OpenPulse admits for a negative on the held-out events against its primary sources, and reports the under-inclusive rate per basis. D2 checks every curated identity mapping that a decisive verdict relies on, and reports the share that are wrong or stale. Both are reported with exact intervals. They are not tests, since no data support a threshold, but H1 is not reported as safety evidence without them, and every under-inclusive admitted declaration is reported beside H1. Their analysis code exists; no annotator record does, so no result exists yet. The completeness ablation is reported as the decisions it changes, split into false clears avoided and correct negatives withheld, per basis. The encoder's curation time per declaration is not measured; whether to add it is open.

Safety reporting. Each policy's false-clear share is reported per event, with an exact bound, and per pair, with an exact bound that treats pairs as independent and an event-clustered bootstrap bound that is withheld when no false clear is observed. Zero observed false clears is reported as an observation, never as evidence that the rate is below a bound. Every result is also reported per completeness basis and per identity status.

Population and stopping (§6–§7). Events are sampled from sources that do not feed OpenPulse, over an observation window that starts after the implementation freeze. An event is included only if a primary source states the change and its scope, it is first observed in the window and archived then, its identifiers are machine-readable, it is found outside OpenPulse's own feeds, and a participating inventory references a project or artefact it names (found by a name search, not a policy run). Events already studied, events whose extent no source defines, and events any policy has seen before their case file is frozen are excluded. No class contributes more than a quarter of the events. Collection stops at 80 included events; if fewer than 60 are at risk, it continues until 60 are. Being at risk is read from adjudicated labels, and no policy runs before collection stops, so the stopping rule never depends on a result. At 80 events, one false-clear event still fails H1 (bound 0.058). Pairs are drawn by stratified random sampling over event class and reference kind.

Leakage control, order and independence (§10). The implementation is frozen at a tag before labelling, with the event definitions, matching rules, policy code, encodings and the frozen catalogue snapshot that every K cell, (v) and OpenPulse read. The reference labels are frozen independently, before any method runs on the held-out pairs. Held-out data come from a later window and from projects outside the development set and the ledger. Fixes made after the freeze are reported as a separate run. The order is: the conformance suite (done; Section 6); then the applicability comparison of Table 21, with (iv) as the primary comparator, X_K in S3 and (v) in S1. The annotators, the adjudicator and the encoder must not be the people who develop the detector.

Studies not registered. The designs of the specification conformance work that remains, of discovery, of the agent decision experiment, and of cost and scale measurement are in `evaluation/FURTHER_STUDIES.md`. They are not part of the registration.

G Normative Membership Table

Table 29 restates the membership tests of Section 5.3 for every supported element form, role and purpose. It was derived from the specification and checked against `core/risk/match.py` and `core/risk/check.py` at v0.8.0. A cell marked ‡ is one where the release departs from it: D16 for untrusted mappings, and the latent package and source-repository gap (Appendix B). The code is not changed in this revision.

Form	Role	Purpose	Member	Not a member (same kind)	Other reference kind
Project p	envelope	bound	P	0 if curated VERIFIED to $q \neq p$; else ? ‡	same rule
Release line (p, ℓ)	exact	bound	M	N outside ℓ; ? not comparable or no version; identity as for a project ‡	same rule
Package, bare n	exact	bound	M	? (names not fully canonical); 0 if curated as another project ‡	image: by identity ‡
Package, purl	exact	bound	M	N other ecosystem; ? same ecosystem; 0 curated other	as bare
Repository r	exact	bound	M (any tag, digest)	N	package: 0; ? if it resolves to p
Tag $r{:}t$, $r{:}\langle pred\rangle$	exact	bound	M	N; ? if d pins only a digest	as repository
Namespace $g/n/*$	envelope	bound	?	N	as repository
Namespace $g/n/*{:}\langle pred\rangle$	exact	bound	M	N; ? digest only	as repository
Object $r@h$	exact	bound	M (equal digest)	? same repository (interim); N other repository	as repository
Registry g	envelope	bound	?	N	package: 0; P if it resolves to p (code ?)
Source repository	exact	bound	–	–	Go module: ? ‡; other package: 0
Repository, tag, object	as form	example	as form	? same repository; 0 other repository	package: 0
Source repository	exact	example	–	–	Go module: M at or below its path (D-R1), else ? ‡; other package: 0

Table 29: Membership tests by element form, role and purpose. Only N is an exclusion; it needs a bounding element and an admissible completeness state. 0 from identity occurs only in the identity-defined rows (project, release line, bare package). A location element is decided by location, whatever d's project. ‡: the release departs from the table (Appendix B). "code ?": the 72-case label difference of Section 6.